



KCensus: Synthesizing Latency-Optimal Consensus Fast Paths [Extended Version]*

Clément Burgelin
EPFL, Lausanne,
Switzerland

Antoine Murat
EPFL, Lausanne,
Switzerland

Gal Sela
EPFL, Lausanne,
Switzerland

Marcos K. Aguilera
NVIDIA, Santa Clara,
USA

Rachid Guerraoui
EPFL, Lausanne,
Switzerland

Abstract

Strongly consistent geo-replication often relies on fast paths to reduce latency in the common case of no failures or contention. Existing fast-path schemes, however, are ad hoc and restrictive: each corresponds to a point in a broad design space shaped by network topology, workload, and latency objective, so no single scheme works best across settings. This paper looks at fast-path schemes from a new perspective, as mechanisms that spread knowledge about proposals. With this view, we identify a fundamental condition on the spread of knowledge for a fast-path scheme to work. We then introduce KCensus, a framework that turns this condition into an optimization problem, synthesizing new fast-path schemes that are optimal for a given setting. We use KCensus to build a geo-replicated key-value store and evaluate it across AWS regions. Our system outperforms competing protocols, with up to 16% lower average latency.

1 Introduction

Modern Internet services rely on geo-replication to remain available despite regional outages. Using consensus for this replication provides the strong consistency many applications require [43, 81]. Consensus, however, adds latency to operations because processes must communicate to safely agree on their ordering [28]. This cost is particularly high in geo-replicated systems, where communication incurs large wide-area delays. At the same time, every millisecond matters for Internet services [61, 72]. Consequently, much work has focused on reducing consensus latency, from Paxos variants to more recent leaderless and topology-aware protocols [4, 9, 10, 55, 57, 65, 68, 78, 85, 91].

These protocols typically use a *fast path* to reduce latency in common executions without failures or contention [57, 68, 78, 91]. Existing fast paths, however, have two drawbacks.

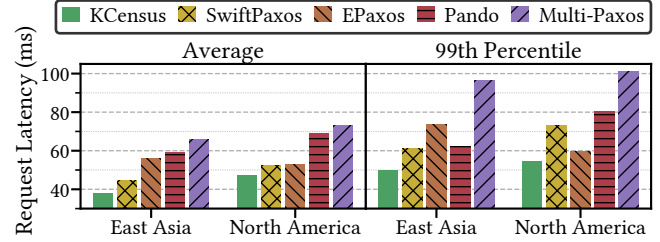


Figure 1. Average and tail latency in two deployments. The best prior protocol changes with the latency metric and topology. KCensus always performs best for the targeted metric.

First, they are complex and error-prone, as subtle concurrent scenarios have led to buggy correctness arguments [80, 86]. Second, they are restrictive: each hard-codes a communication pattern, quorum criteria, and recovery mechanism. Yet no fixed choice is optimal: the best one depends on the network topology, the proposer, and the target latency metric.

Figure 1 illustrates this effect: among existing protocols, the one with the lowest average latency does not always have the lowest tail latency, and the tail-latency winner changes across deployments. More fundamentally, using one fast-path strategy for all proposers is itself too restrictive. A proposer’s fast-path *strategy* specifies what it must know about other processes’ states before deciding. Best performance may require different strategies for different proposers, tailored to their location and proposal rate. These strategies must nevertheless coexist safely: a decision made with one must preclude a conflicting decision made with another.

The key challenge is recovery. If a fast path fails, the fallback must preserve any decision that may already have occurred. Thus, different possible fast decisions must remain distinguishable after failures. We formalize this as a general *recoverability* property of consensus: *two reachable system states in which different values were decided must differ in the local states of more than f processes*, where f is the number of tolerated crashes. Otherwise, after those processes crash, the survivors cannot distinguish the two states.

To translate this property into fast-path design, we view fast paths as mechanisms that spread evidence about acceptances (i.e., votes): which processes accepted a value and who observed those acceptances. Specializing the recoverability

*Extended version of the paper to appear in the 22nd European Conference on Computer Systems (EuroSys ’27). Proceedings version: <https://doi.org/10.1145/3842654.3848528>. © 2027 Copyright held by the owner/author(s). Licensed under CC BY 4.0.

property yields two fundamental conditions: *every fast-path quorum must contain more than f processes*, and *conflicting quorums must intersect and evidence about the intersection must survive f failures*. We further prove these conditions necessary and sufficient, thereby tightly characterizing how much work a fast path must do.

We use these conditions to build *KCensus* (**K**nowledge **C**ensus), a framework for synthesizing consensus fast paths. KCensus encodes each proposer’s fast-path strategy as a knowledge requirement, reducing correctness to compatibility checks between them, and selects a compatible set of requirements that minimizes the target latency (e.g., mean or tail) for the network topology and proposer distribution.

To demonstrate KCensus’s practicality, we use it to build a strongly consistent geo-replicated key-value store whose fast path is optimized based on network topology, proposer distribution, and target latency metric. In our contention-free experiments, our system always achieves the best targeted metric—cutting average latency by up to 16%. This shows that synthesized fast paths are not only cleaner and more flexible than hard-coded ones, but also significantly faster.

In summary, this paper makes the following contributions:

- We formulate a recoverability property of consensus, derive from it the fundamental knowledge conditions that fast paths must satisfy to remain safely recoverable, and prove that they are both necessary and sufficient.
- We present KCensus, a knowledge-based framework for expressing and verifying per-proposer fast-path strategies.
- We show how to synthesize an optimal fast path from network topology, proposer distribution, and target latency metric, thereby uncovering new fast-path strategies.
- We implement KCensus in a geo-replicated key-value store, open-sourced at github.com/LPD-EPFL/kcensus, and evaluate it across geo-distributed AWS deployments, showing substantial latency improvements over prior protocols.

2 Background

We review consensus-based strong geo-replication (§2.1), explain how prior protocols improve latency through fast paths using adopt-commit (§2.2), and state our system model (§2.3).

2.1 Replication and consensus

State Machine Replication (SMR) [81] is a standard technique to build reliable services with strong consistency, i.e., linearizability [43]. Replicas start from the same initial state and apply the same stream of deterministic commands, thereby giving clients the illusion of a single fault-tolerant server. Replicas agree on this stream by repeatedly running a consensus protocol, typically once per stream slot [28]. In each consensus instance, processes propose *values*, no two decide differently, and correct processes eventually decide.

2.2 Fast paths and adopt-commit

Consensus protocols reduce latency by optimizing a common-case *fast path*, typically for executions without failures or contention. Multi-Paxos-style stable-leader protocols keep this path simple: a leader decides values by contacting a majority [55, 92]. This is fast for clients near the leader, but its latency depends on leader placement. Other protocols make different fast-path choices. Fast Paxos and EPaxos allow multiple proposers to use the fast path concurrently [57, 68], while topology-aware protocols tailor its communication pattern to wide-area deployments [4, 78, 91]. For our purposes, these fast paths differ mainly in *which* processes participate and *how* knowledge flows among them, which dictates their performance for a given setting.

As previously observed [54, 76], a fast path can be modeled as an *adopt-commit* protocol [93]. Processes propose values as in consensus, but a proposer returns either *commit*(v) or *adopt*(v). Committing v means that v can be decided immediately: the fast path succeeded, and no different value may be adopted or committed. Adopting v means that the proposer could not commit on the fast path, but v may have committed elsewhere. The proposer therefore carries v into a fallback consensus protocol, such as Paxos, ensuring that recovery preserves any fast-path decision.

Formally, adopt-commit satisfies three properties: (1) **Termination**: every correct proposer eventually adopts or commits; (2) **Validity**: only proposed values are adopted or committed; and (3) **Agreement**: if v commits, then only v may be adopted or committed. Note that, in our formulation, not all correct processes need to propose.

Building on this abstraction, KCensus synthesizes optimized adopt-commit protocols with per-proposer strategies.

2.3 System model

We consider a standard point-to-point message-passing system with $n \geq 2f+1$ processes, of which up to f may crash. Links are FIFO and lossless between correct processes.

The adopt-commit protocols synthesized by KCensus, and the SMR protocol built atop them, are safe under full asynchrony. Our adopt-commit protocols need eventual crash suspicion to terminate, while good performance requires no false suspicions during stable periods [16]. As usual, we rely on partial synchrony for deterministic SMR termination [28].

For synthesis, we also assume stable periods long enough to estimate inter-process latencies and proposal rates, and install new strategies (§6). This assumption is needed only for optimization, not safety.

3 Recoverable Fast Paths

This section characterizes what information a fast path must leave behind for safe recovery (§3.1), and shows how the resulting conditions explain familiar protocols (§3.2).

3.1 Recoverability

Intuitively, safe recovery requires different possible commits to remain distinguishable after failures. We formalize this intuition with a new *recoverability* property of adopt-commit:

Any two reachable states in which different values were committed must differ in the local states of more than f processes.

Suppose otherwise that states S_1 and S_2 , each reached by some execution [36], contain commits of v_1 and $v_2 \neq v_1$, but differ at no more than f processes. If these processes crash and their pending messages are lost, the survivors have identical local states in both executions. They therefore behave identically under the same continuation, yet if a survivor proposes, Termination requires that it eventually adopt or commit, while Agreement allows only v_1 after S_1 and only v_2 after S_2 , a contradiction. By the same reasoning, consensus satisfies an analogous recoverability property, with *decided* in place of *committed*.

Knowledge requirements. We specialize this property to *acceptance-based* fast paths, which are algorithms in which each process accepts the first value it sees in an adopt-commit instance, never accepts another, and all tracked knowledge concerns these acceptances. We say that a process *witnesses* an acceptance when its local state records it.

We represent proposer p 's strategy by a *knowledge requirement* R_p , which is a mapping where $R_p.\text{keys}()$ are the *required witnesses* of p , and for each $q \in R_p.\text{keys}()$, $R_p[q]$ is the set of processes that q must have recorded as acceptors of p 's value. We say that R_p is *met* once p knows that each such q holds these records.

For example, if $R_p = \{p:\{p, q, r\}, q:\{p, q\}, r:\{p, r\}\}$, then p may commit its value v only after it knows that (1) p , q , and r accepted v ; (2) q witnessed the acceptances of p and q ; and (3) r witnessed the acceptances of p and r .

Because each process first witnesses its own acceptance, we assume without loss of generality the following normal form: for every $q \in R_p.\text{keys}()$, $R_p[q] \subseteq R_p.\text{keys}()$ and $q \in R_p[q]$. Indeed, by making every recorded acceptor a required witness and every required witness a witness of its own acceptance, any requirement can be put into this form without increasing fast-path latency. By the normal form, the required witnesses $R_p.\text{keys}()$ are exactly the processes whose acceptances must be recorded; we thus call this set the *quorum* Q_p of R_p .

A proposer may hold several strategies, e.g., one per quorum it may use (§3.2), and commit when any is met; the two conditions below apply to each.

Validity. Compare two executions in which the same proposer p commits different values using R_p . Only the members of Q_p need to have accepted when p commits, so the two executions may differ only at Q_p . Recoverability therefore yields the *validity* condition: *every quorum must contain more*

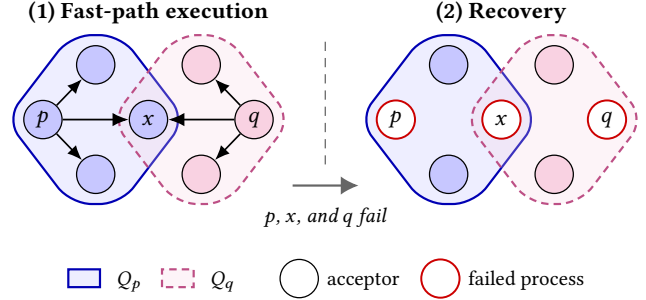


Figure 2. Loss of intersection evidence. (1) x accepts p 's value, allowing p to commit (return messages are omitted). (2) After p , x , and q fail, no survivor records x 's acceptance.

than f processes (i.e., $|Q_p| > f$). We call a requirement that satisfies this condition *valid*.

Compatibility. Consider requirements R_p and R_q for distinct proposers p and q . Their quorums must intersect: otherwise, both requirements could be met concurrently for different values, allowing conflicting commits and violating Agreement. Yet intersection alone is insufficient: recoverability requires *evidence*—information in local states that distinguishes conflicting commits—to survive f failures.

Figure 2 shows why. Suppose $n=7$, $f=3$, and $Q_p \cap Q_q = \{x\}$. Consider two executions that differ only at p , q , and x . In both, p proposes v_p and q proposes $v_q \neq v_p$. In one execution, x accepts v_p and p commits; in the other, x accepts v_q and q commits (not shown). If p , q , and x then fail, the survivors have identical local states and cannot distinguish the two.

We now ask which processes are guaranteed to distinguish the two executions. Every process in $Q_p \cap Q_q$ does so directly, because it accepts a different value in each execution. A process outside the intersection is guaranteed to distinguish them only if R_p or R_q requires it to witness an acceptance in the intersection before the corresponding commit. Recoverability therefore yields the *compatibility* condition: *every pair of requirements R_p and R_q must together require more than f processes to witness an acceptance by some process in $Q_p \cap Q_q$* . The condition is symmetric, but permits asymmetric evidence: one requirement may contribute most witnesses. We call two requirements satisfying this condition *compatible*.

For example, p , q , and x in Figure 2 provide three distinguishing witnesses. If R_p also requires another process w to have recorded x 's acceptance (i.e., $x \in R_p[w]$), then w becomes a fourth witness, making R_p and R_q compatible for $f=3$. Each witness w that R_p requires costs a causal chain: from p to some $x \in Q_p \cap Q_q$, then to w , and back to p .

Together, validity and compatibility exactly characterize correct fast paths under our model. Both conditions are necessary for any correct fast path, even in protocols not defined in terms of acceptance. Intuitively, for each proposer we can

represent the causal message chains preceding its fast decision as a knowledge requirement. Because the protocol must satisfy the recoverability property, these induced requirements satisfy both conditions. Conversely, any collection of requirements satisfying both conditions admits safe recovery. We prove the necessity of both conditions in Appendix C (Theorem C.2), and Sections 4 and 5 demonstrate their sufficiency.

3.2 Prior protocols and recoverable evidence

Prior protocols can be understood through the knowledge requirements induced by their common-case communication. Each protocol differs in where the evidence required for compatibility lies, how it is spread, and how it is recovered, yielding different latency profiles across topologies. Figure 3 shows the main cases discussed below.

Paxos, Multi-Paxos, and Pando. These protocols simplify recovery by electing one proposer before values are accepted, so only one path is active at a time. Paxos and Pando perform this election in their respective first phases [55, 91], while Multi-Paxos amortizes it across values [92]. Once a leader is elected, recovery only needs to preserve its value, so any quorum larger than f suffices.

Their induced requirements nonetheless satisfy both conditions. For Paxos and Pando, meeting the induced requirement first entails having a majority accept the value—any two such majorities intersect—and then making a majority witness these acceptances, ensuring that evidence of the intersection survives f crashes. In Multi-Paxos, all quorums intersect at the stable leader, whose acceptance is witnessed by a majority. KCensus’s optimal fast path is therefore no slower than these protocols’ common-case paths.

Fast Paxos. Fast Paxos allows concurrent proposers to commit in one round trip [57]. Because there is no second round to spread evidence, the processes in the intersection of two quorums must themselves provide the $f+1$ witnesses required for recovery. Moreover, Fast Paxos does not require proposers to commit to quorums in advance: a proposer may commit with any sufficiently large set of acceptors. Thus, every two possible fast quorums of size q must intersect in more than f processes, requiring $2q - n > f$, or $q > (n+f)/2$. For example, with $n=7$ and $f=3$, a fast quorum needs 6 processes, rather than the minimum $f+1=4$ (Figure 3(a)).

SwiftPaxos. SwiftPaxos also supports multiple concurrent proposers and admits two kinds of fast paths [78]. In one mode, it uses large quorums à la Fast Paxos. In another, all proposers share a single optimally small quorum of size $f+1$ (Figure 3(b)). Either way, intersections themselves are large enough for direct recovery. Moreover, since all quorums intersect at a designated leader, these requirements are compatible with Multi-Paxos requirements that use it.

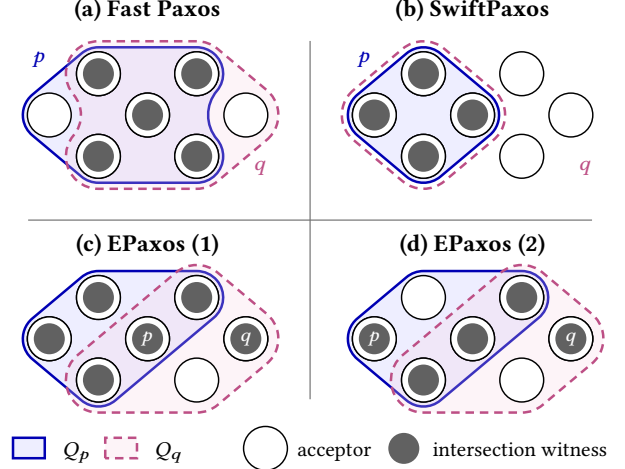


Figure 3. Intersection witnesses in prior protocols for $n=7$ and $f=3$. The witnesses of Fast Paxos and SwiftPaxos lie directly in quorum intersections, while EPaxos also uses witnesses outside them because each proposer both witnesses its quorum and is witnessed by it.

EPaxos. EPaxos also commits in one round trip, but unlike Fast Paxos and SwiftPaxos, each proposer is itself an acceptor and belongs to its own fast quorum [68]. This lets the protocol create witnesses outside the quorum intersection. For example, with $n=7$ and $f=3$, an EPaxos fast quorum has 5 processes, so two quorums may intersect in only 3—fewer than the $f+1=4$ witnesses required for recovery. If the intersection contains proposer p , then p ’s acceptance lies in the intersection; because p accepts before sending its value, every process in Q_p witnesses that acceptance (Figure 3(c)). Otherwise, the intersection excludes both proposers: its members witness their own acceptances, while the replies make p and q additional witnesses, yielding at least $|Q_p \cap Q_q| + 2 > f$ witnesses (Figure 3(d)). Thus, EPaxos spreads evidence beyond the intersection within the same round trip, allowing smaller intersections than Fast Paxos.

In practice, SwiftPaxos and EPaxos do not decide one command per slot, but dependencies for each command. This distinction is not fundamental. Their correctness relies on *Visibility* and *Agreement* [80]: for any two conflicting commands, at least one must depend on the other, and all processes must agree on each command’s dependencies. Thus, for conflicting commands c_p and c_q , deciding whether c_q belongs to c_p ’s dependencies is itself consensus and must remain recoverable. Their fast paths therefore satisfy the same validity and compatibility conditions.

These examples show that familiar protocols satisfy the same recoverability conditions through different evidence patterns, yielding different latency profiles across topologies. KCensus captures these patterns as per-proposer knowledge requirements, which instantiate a generic fast-path template.

Algorithm 1. Valid and Compatible Requirements

```

1 def valid(req): return |req.keys()| > f
3 def compatible(req_a, req_b):
4     intersection = req_a.keys() ∩ req_b.keys()
5     // Acceptors of A/B that must witness the inter.
6     witnesses_a = {p s.t. req_a[p] ∩ intersection ≠ ∅}
7     witnesses_b = {p s.t. req_b[p] ∩ intersection ≠ ∅}
8     return |witnesses_a ∪ witnesses_b| > f

```

Algorithm 2. KCensus’s Adopt-Commit’s Propose Method

```

9 def AdoptCommit::Propose(v):
10 send Accept&Spread(v, {}) to myself // init spread
11 wait until can_commit(v) or DoAdopt() was received
12 if wait was ended by can_commit(v):
13     trigger AdoptCommit::Commit(v)
14 else: // DoAdopt() was received, recovery
15     send Freeze() to all // including self
16     wait for n-f Frozen(*) replies
17     v' = potential_commit(replies)
18     if v' == ⊥: v' = v // any proposed value works
19     trigger AdoptCommit::Adopt(v')

```

4 KCensus’s Fast-Path Template

KCensus turns the recoverability conditions of Section 3 into a generic adopt-commit template. This template is parameterized by a knowledge requirement per proposer. This knowledge requirement is checked for validity and pairwise compatibility (§3.1) by Algorithm 1. Section 5 explains how to select an optimal combination of requirements for a given deployment.

The algorithms below describe a single instance of a one-shot adopt-commit protocol—our SMR uses one such instance as the fast path for consensus in each log slot (§6).

After an overview (§4.1), we describe evidence dissemination (§4.2), commitment (§4.3), recovery (§4.4), and non-voting proposers (§4.5).

4.1 Overview

Algorithm 2 gives the top-level Propose method. To propose a value v , process p starts disseminating evidence about v ’s acceptance by invoking its own Accept&Spread handler (line 10). It commits once it learns enough evidence to meet its requirement R_p (lines 11–13); a conflict or fault suspicion instead abandons the fast path and triggers recovery (line 14).

Recovery begins by freezing $n-f$ processes (lines 15–16). Because every valid requirement has more than f witnesses, the frozen set contains at least one from each requirement. Once frozen, such a witness cannot gain evidence, so its Frozen reply either rules out any commit requiring evidence the witness lacks, or records the evidence a commit could rely on. These replies form the **Knowledge Census** that gives KCensus its name. From this census, p determines what value,

Algorithm 3. Evidence Spread

```

20 accepted = ⊥ // local
21 known_acceptors: array<set<pid>> = [∅] × n // local
23 upon recv. Accept&Spread(v, knowledge_of_q) from q:
24     if accepted == ⊥: accepted = v
25     if accepted ≠ v: // conflict detected
26         send DoAdopt() to all // including self
27     else:
28         old_acceptors = known_acceptors
29         known_acceptors[me] ∪= {me} ∪ knowledge_of_q[q]
30         for each process r:
31             known_acceptors[r] ∪= knowledge_of_q[r]
32         if known_acceptors ≠ old_acceptors:
33             send Accept&Spread(v, known_acceptors) to others
35 upon receive Freeze() from q:
36     stop handling Accept&Spread(*) messages
37     send DoAdopt() to all // including self
38     send Frozen(me, accepted, known_acceptors[me]) to q

```

if any, can be a possible fast commitment (line 17). If this value exists, p adopts it (line 19); otherwise, any proposed value is safe, so p adopts its own (line 18).

4.2 Disseminating evidence

Algorithm 3 disseminates evidence as quickly as possible. A process accepts the first value it receives and records it in accepted (line 24). It never changes its acceptance. If it later receives evidence for another value, it detects a conflict and asks proposers to abandon the fast path (lines 25–26).

The known_acceptors array records witnessed evidence. At each process, when accepted = v , known_acceptors[q] represents its current view of which processes q has witnessed accepting v . Thus, known_acceptors[me] is the process’s own first-order evidence, while other entries are its second-order view of other processes’ evidence. Whenever this evidence grows, the process broadcasts the updated array (lines 32–33). Every acceptance represented in known_acceptors[q] originates in a report from q , even if relayed by other processes; consequently, this entry attributes to q only acceptances that q has witnessed.

Without conflicts, evidence therefore propagates in three conceptual steps: (1) processes accept the value, (2) processes learn which acceptances occurred, and (3) the proposer learns what evidence each witness can report. Figure 4 illustrates this propagation in a five-process ring.

Upon receiving Freeze, a process stops handling further Accept&Spread messages for any value (line 37) and replies with its accepted value and known_acceptors[me] (line 38). Because a process’s own entry only grows until it freezes, the entry in its Frozen reply is a superset of every earlier version, including any version on which a proposer may have relied.

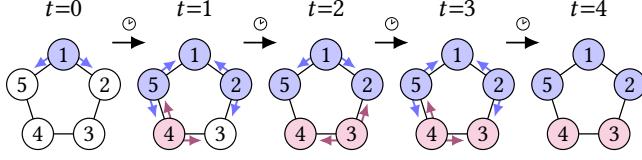


Figure 4. Evidence spread in a five-process ring. Different colors represent different accepted values, and arrows represent broadcasts. At $t=0$, P1 starts spreading its value, which reaches P2 and P5 at $t=1$ and is relayed onward. At the same time, P4 spreads its own value. At $t=2$, P3 accepts P4's value, so P3 and P4 ignore P1's. Processes broadcast until $t=4$. Tables 1 and 2 show the resulting evidence.

Table 1. Processes' 1st-order evidence: $\text{known_acceptors}[me]$

Process	$t=0$	$t=1$	$t=2$	$t=3$	$t=4$
P1	{1}	{1}	{1,2,5}	{1,2,5}	{1,2,5}
P2	$\emptyset$	{1,2}	{1,2}	{1,2,5}	{1,2,5}
P3	$\emptyset$	$\emptyset$	{3,4}	{3,4}	{3,4}
P4	$\emptyset$	{4}	{4}	{3,4}	{3,4}
P5	$\emptyset$	{1,5}	{1,5}	{1,2,5}	{1,2,5}

Table 2. Proposers' 2nd-order evidence: known_acceptors

Process	$t=0$	$t=1$	$t=2$	$t=3$	$t=4$
P1	1:{1}	1:{1}	1:{1,2,5} 2:{1,2} 5:{1,5}	1:{1,2,5} 2:{1,2} 5:{1,5}	1:{1,2,5} 2:{1,2,5} 5:{1,2,5}
P4	$\emptyset$	4:{4}	4:{4}	3:{3,4} 4:{3,4}	3:{3,4} 4:{3,4}

Table 3. Potential commits P2 computes at $t \in [0, 4]$ for each set of $n - f$ replies, based on Table 1.

Replies	$t=0$	$t=1$	$t \in [2, 4]$
Includes P1's	$\perp$	$\perp$	P1's value
Excludes P1's	$\perp$	P1's value	P1's value

4.3 Committing a value

Algorithm 4 checks whether R_p is met: after verifying that it accepted v itself, proposer p checks for every key q that its current view of q 's evidence covers $R_p[q]$ (lines 43–46).

For example, return to the execution in Figure 4. Suppose P1 uses requirement $\{1:\{1,2,5\}, 2:\{2\}, 5:\{5\}\}$, while P4 uses $\{4:\{3,4,5\}, 3:\{3\}, 5:\{5\}\}$. By Table 2, P1 meets its requirement at $t=2$, one round trip after proposing, and may commit. P4 never meets its requirement.

Failures can prevent required evidence from arriving. A proposer therefore abandons the fast path if it suspects one of its required witnesses (Algorithm 5). As discussed in §2.3, a false suspicion is safe: it merely triggers recovery earlier.

Algorithm 4. KCensus's Safe Commitment

```

39 REQUIREMENTS[me]: dict<pid, set<pid>> // constant
40 // Check if known_acceptors covers my requirement
41 def can_commit(v):
42     if accepted  $\neq$  v: return False
43     for q in REQUIREMENTS[me].keys():
44         if REQUIREMENTS[me][q]  $\not\subseteq$  known_acceptors[q]:
45             return False // Missing evidence
46     return True // Requirement fulfilled

```

Algorithm 5. Fast-Path Abandonment on Suspicion

```

48 upon event WeakFailureDetector::PotentialFailure(q):
49     if q in REQUIREMENTS[me].keys():
50         send DoAdopt() to myself

```

Algorithm 6. KCensus's Value Adoption

```

51 def potential_commit(replies):
52     for each value  $v \neq \perp$  in replies:
53         if could_potentially_commit(v, replies): return v
54     return  $\perp$  // No value could commit
55
56 def could_potentially_commit(v, replies):
57     req_v = REQUIREMENTS[v.proposer]
58     for each Frozen(p, val $\neq$ v, acceptors) in replies:
59         if ( $\{p\} \cup \text{acceptors}$ )  $\cap$  req_v.keys()  $\neq \emptyset$ :
60             return False // Missing required acceptors
61     for each Frozen(p, val==v, acceptors) in replies:
62         if p in req_v.keys() and req_v[p]  $\not\subseteq$  acceptors:
63             return False // p lacks evidence about v
64     return True // Cannot rule out commit

```

4.4 Adopting a value

When a proposer cannot commit, it freezes $n - f$ processes and treats their Frozen replies as a census of the recoverable evidence. Algorithm 6 tests each value v appearing in this census against the requirement of v 's proposer. A reply rules out v in either of two ways: it shows that some required acceptor accepted a different value (lines 58–60), or the reply comes from a required witness that accepted v but lacks evidence required by R_v (lines 61–63). Any value not ruled out remains a possible fast commitment.

The key safety invariant is: *if a value v committed, then every set of $n - f$ Frozen replies preserves v and rules out every conflicting value.* Validity ensures that more than f required witnesses accepted v , so every census contains a reply carrying v . Because R_v was actually met, no reply can rule v out. Now consider a conflicting value v' . Compatibility gives more than f witnesses, required by v or v' , that observed an acceptance in the intersection of their quorums. Therefore, every census contains at least one such witness. Its report either reveals that an acceptor required by v' chose v , or shows that a required witness for v' lacked evidence that $R_{v'}$ would have demanded. Thus, every conflicting v' is ruled out. If some value committed, it is therefore the only surviving

candidate and must be adopted. This realizes the sufficiency direction of the conditions in §3.1, of which a full proof is given in the appendix.

As a concrete example, return to Figure 4. Suppose a conflict or failure suspicion triggers recovery at P2 at some $t \in [0, 4]$, and P2 instantaneously receives Frozen replies from any $n-f=3$ processes.¹ Table 3 lists the resulting potential commit for every such reply set, grouped by whether it includes P1. At $t=1$, P1’s reply rules out its value, leaving no potential commit ($\perp$). This is because its requirement calls for evidence of P2’s and P5’s acceptances, which P1 has not yet witnessed. Without P1’s reply, recovery cannot observe this missing evidence, so P1’s value remains possible. From $t=2$, when P1 can commit, P2 always adopts P1’s value, regardless of which three processes reply.

4.5 Non-voting processes

So far, every proposer is a voting process whose failure counts toward the failure budget f . In geo-replicated systems, clients benefit from proposing directly, which saves a hop to a replica. Yet we should not include clients as regular proposer processes in the protocol, because doing so disrupts the resiliency of the system. Indeed, clients are numerous and flaky—more than a majority may crash—so adding them as regular processes could cause executions to exceed the failure budget. KCensus thus lets clients act as *non-voting processes* that merely propose values and disseminate evidence.

A non-voting process never appears as a required witness or acceptor in any requirement. No commit therefore depends on its state, so its failure does not count toward the failure budget.

Non-voting processes otherwise run the same protocol, with two exceptions. Their evidence is not tracked: they do not add {me} in line 29, and line 30 ranges only over voting processes. They also ignore Freeze messages (line 35), since recovery needs reports only from voting processes. Accordingly, the n used in line 16 counts only voting processes.

5 Synthesizing Optimal Fast Paths

The template of Section 4 works with any valid and compatible requirement combination. This section explains how KCensus finds one that minimizes the target latency metric for a given deployment, both initially and when conditions change (§6). We first turn topology measurements into candidate requirements (§5.1), then choose a compatible combination that optimizes the target metric (§5.2), and finally reduce network traffic without affecting latency (§5.4).

5.1 From topology to requirements

The first step is to discretize the design space. Although time is continuous, a proposer’s requirement can change only

¹After the freeze, this hypothetical execution diverges from Figure 4.

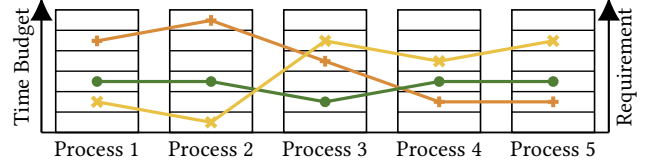


Figure 5. Requirements optimization. Each column lists one proposer’s time budgets; each cell represents a budget and the requirement the proposer can meet within it. Lines show compatible combinations with one requirement per proposer. The optimizer picks the best line for the objective.

when the proposer learns new evidence. Thus, each evidence arrival defines a relevant *budget*: the earliest time at which the proposer can satisfy a stronger requirement.

For each proposer p , KCensus simulates a single-proposer execution of the evidence-spreading algorithm from Section 4.2, using measured latencies. The simulation records when p learns each piece of first- or second-order evidence. For every resulting budget b , KCensus derives the requirement $R_{p,b}$, i.e., the pieces of evidence that p can gather by time b . Every candidate has the normal form of §3.1, which Algorithm 3 ensures by construction (lines 29–31). Lastly, invalid requirements are discarded.

This step replaces an intractable search over arbitrary requirements (up to $2^{n \times n}$ combinations) with a search over the budgets at which the proposer learns something new. The derived requirements may include unnecessary evidence: for example, a 10 ms budget also includes evidence that arrived after 1 ms, even if compatibility does not need it. This extra evidence does not increase latency and can later be trimmed.

5.2 Choosing optimal requirement combinations

Given the candidate budgets, KCensus selects one per proposer. Figure 5 visualizes this search: a choice is feasible only if the selected budgets induce compatible requirements.

Problem 1 gives the average-latency version of the optimization problem. Here, B_p is the set of budgets available to proposer p , $R_{p,b}$ is the requirement p can meet within budget b , and P_p is the probability that p proposes. The objective minimizes expected fast-path latency, while the constraints enforce pairwise compatibility (§3.1).

Problem 1. Average-latency optimization problem

- B_p : time budgets available to process p .
- $R_{p,b}$: requirement process p can meet with budget b .
- P_p : probability that process p is the proposer.

$$\mathbf{b}^* = \arg \min_{(b_1, \dots, b_n) \in B_1 \times \dots \times B_n} \sum_{p=1}^n P_p b_p$$

s.t. $compatible(R_{p,b_p}, R_{q,b_q}) \quad \forall p, q \in \{1, \dots, n\}.$

By changing only the objective, KCensus can also target median or tail latency while keeping the same compatibility constraints. Because compatibility couples proposers' budgets, different objectives may select different Pareto-optimal solutions; across our evaluated deployments, their number ranges from one to over a hundred. Whatever the objective, any feasible solution instantiates the same template and is correct by construction.

To make the search fast, KCensus prunes *dominated* budgets. A budget is dominated if it is no cheaper for the objective and does not enable compatibility with any requirement that a cheaper budget cannot already support. Concretely, for each candidate requirement, KCensus computes the cheapest requirements of other proposers that are compatible with it, and removes candidates that are never minimal for any counterpart. This pruning is effective: with up to 31 AWS replicas, it leaves only up to 10 candidate budgets per proposer, which a branch-and-bound solver explores quickly (§7.7).

The optimizer also accounts for suspected failures. It excludes candidate requirements that depend on suspected processes. A suspected proposer may nevertheless still be correct and propose, so it receives a requirement guaranteed compatible with every requirement the optimizer may select.

Optimality. The optimizer yields latency-optimal fast paths under fixed and known network latencies, permanent crashes with perfect failure detection, contention-free executions, and a fixed latency metric. A proof appears in the appendix. Briefly, for each proposer p and budget b , the simulation computes all evidence that p can obtain by time b under the given topology. Any deterministic fast path committing by b can therefore rely only on evidence contained in this snapshot. If the requirements induced by a vector of budgets are incompatible, committing within those budgets would violate the recoverability conditions of Section 3, which every consensus protocol must satisfy. Conversely, every compatible vector can instantiate the template of Section 4. Hence, under these assumptions, no deterministic consensus fast path can have lower latency according to the target metric.

5.3 Robustness with multiple requirements

For simplicity, the presented KCensus template assigns a single requirement to each proposer, and the proposer abandons the fast path upon suspecting a required witness (Algorithm 5) so as not to wait indefinitely. A proposer can instead maintain several requirements, commit when any is met, and abandon only when all are ruled out. We leave optimizing such sets for robustness to future work, but observe that Paxos-like requirements with quorums of $n-f$ processes (§3.2) are compatible with every valid requirement. Adding one for every such quorum therefore bounds conflict-free latency by that of Paxos, without needing a timeout, at the cost of extra messages.

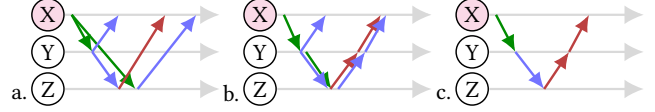


Figure 6. Accept&Spread messages satisfying X 's requirement $\{X: \{X, Y, Z\}, Y: \{X, Y\}, Z: \{X, Y, Z\}\}$: (a) as induced by the requirement, (b) with latency shortcuts, and (c) without redundant messages. Colors mark the newest evidence each message adds: X 's acceptance (green), Y 's or Z 's acceptance and record of X 's (blue), and Z 's record of Y 's (red).

5.4 Reducing network traffic

Algorithm 3 broadcasts evidence whenever it grows. This propagates evidence as quickly as possible, but may disseminate evidence that no selected requirement needs. Once KCensus has chosen a requirement combination, it replaces these broadcasts with a dissemination graph that preserves commit latency while reducing communication.

Consider the requirement in Figure 6. To satisfy it, X must learn that X , Y , and Z accepted the value, that Y witnessed X 's and Y 's acceptances, and that Z witnessed all three. These dependencies directly induce a communication graph sufficient to satisfy X 's requirement, shown in Figure 6(a).

KCensus then optimizes this graph. First, it uses latency shortcuts: measured wide-area latencies need not satisfy the triangle inequality, so relaying evidence from X to Z through Y , for example, may be faster than sending it directly. KCensus uses such shortcuts when they reduce latency (Figure 6(b)). If a shortcut relies on a process outside the requirement, the failure detector also monitors that process so that its failure triggers fast-path abandonment.

Second, KCensus removes redundant messages. For example, if Z reports to X after learning that Y accepted, an earlier message from Z reporting only its own acceptance may be unnecessary. Likewise, if X learns Y 's acceptance through Z , Y need not also report it directly to X . Figure 6(c) shows the resulting graph.

All processes deterministically compute the same dissemination graph from the requirements and latency table. The graph specifies when and where each piece of evidence is sent and also enables compact messages: rather than carrying an explicit set of known acceptors, a message can identify the graph edge it traverses, from which the receiver infers the corresponding evidence.

Accept&Spread messages can also carry only a short proposer identifier rather than the full value. Replicas buffer such evidence until they receive and log the corresponding value, preventing a commit whose payload is unavailable.

Overall, requirements determine *what* evidence a commit needs, while the dissemination graph determines *how* that evidence is delivered. Because the graph changes only the delivery—not the requirements, when they can be met, or the commit checks—commit latency and safety are unchanged.

6 Implementation: KCensus-Based SMR

We implement KSMR, a replication engine that uses KCensus-synthesized fast paths to build SMR logs (§2.2). Each voting process hosts a service replica and a client; other clients run as non-voting processes (§4.5). Our implementation consists of 8,764 lines of Rust code and includes the KSMR runtime, a CPU solver that finds optimal requirements (§5.2), and a strongly consistent geo-replicated key-value store built on KSMR.

In a nutshell. KSMR runs one consensus instance per log slot. Each instance’s fast path is optimized for the current topology, proposer rates, and latency objective (§5). A process proposes its command to the first slot s it believes is empty. For slot s , KSMR instantiates KCensus’s adopt-commit template with the current requirements: a fast commit triggers immediate decision in the slot, while a conflict or fault triggers adoption of a safe command, which is passed to fallback consensus (§2.2). If a process fails to commit in slot s , it re-proposes its command in slot $s+1$ and repeats until it commits. Processes announce commits, allowing replicas to execute promptly and proposers to track empty slots.

Conflicts. Upon conflicts, KSMR falls back to Paxos [55]. While Paxos takes two roundtrips to a majority, the first constitutes a mere leader-election phase (§3.2): KSMR overlaps this phase with KCensus’s adopt-commit using the same messages. Thus, on a conflict, the highest-id concurrent proposer becomes leader and, after adopting a safe command, only needs to run Paxos’s second phase. Adoption adds no separate round: processes freeze as soon as they observe a conflict or finish their part of the dissemination graph (§5.4). They then keep forwarding the messages prescribed by the graph and piggyback their frozen state on the last one. The leader can therefore complete adoption by the time a fast commit would complete. In practice, fast commits take one roundtrip in most deployments and only slightly more otherwise (§7.1), so conflicts are about as slow as Paxos. Moreover, when the leader knows KCensus’s adopt-commit did not commit (Algorithm 2, line 18), it batches all the commands it knows conflicted into one Paxos proposal.

Sharding. KSMR shards requests so that non-interfering commands can be replicated concurrently in independent logs without conflicting. For instance, if replicating a key-value store, updates that target different keys never conflict. Additionally, shards use different requirements to account for the distribution of proposers in each of them (§5.2).

Latency monitoring. KSMR is bootstrapped with knowledge requirements based on the initial network latency. Then, in the background, KSMR processes monitor and share inter-process latencies in the form of a latency table. Upon a crash, broken link, or proposal-rate shift, a process proposes an updated table as a cross-shard command. Consensus installs

the same table everywhere; each process then runs the deterministic solver locally to derive the same requirements.

Delegated execution. A non-voting client hosts no service replica, so it would still have to contact a replica to execute its command and reply, adding latency. To avoid this extra step, KSMR can delegate commitment to a nearby replica. The client remains the logical proposer and starts the fast path, but the delegate replica performs the `can_commit` check, commits once the required evidence is available, executes the command, and replies to the client. KSMR then uses requirements synthesized for proposer–delegate pairs.

Reads. Like existing SMR systems, KSMR does not replicate reads. Instead, it executes them at a single replica after checking that its state is up to date [17]. To do so, a read queries a quorum that intersects every write quorum, thereby discovering any write that may have committed. The read then waits until the highest such slot has been executed. Reads thus conflict with writes, but never delay them.

7 Evaluation

We evaluate the fast paths of consensus protocols, comparing KCensus (using our KSMR engine) against prior baselines.

Baselines. As we focus on common-case replication latency, we compare KCensus against protocols spanning the main approaches to reducing SMR latency: a stable-leader protocol, leaderless fast-quorum protocols, and wide-area latency-optimized ones. Our baselines are Multi-Paxos [92], EPaxos [68], SwiftPaxos [78], and Pando [91].

All baselines are re-implemented in KSMR’s Rust codebase, with their fast-path latency validated against the specifications, so language and runtime do not confound our resource comparison (§7.4). Whenever a protocol leaves choices such as leader placement or quorum selection, we use the configuration that minimizes average latency in our deployment. For Pando, this means disabling erasure coding. Moreover, EPaxos runs in non-thrifty mode.

Testbed and configurations. We deploy all systems on AWS using `t3.medium` instances (Intel Xeon Platinum 8000-series), except the conflict and load experiments (§7.5–7.6), which use non-burstable `c6i.large` instances (Intel Xeon Ice Lake). All run single-threaded. All deployments have optimal resilience ($2f+1$ replicas), with one client per region.

Our end-to-end experiments use four 7-replica deployments: three continental deployments in North America (NA), Europe (EU), and East Asia (EA), and one deployment spanning the Northern Hemisphere (NH). Our scalability experiments vary the deployment size from 3 to 31 replicas across AWS regions worldwide.

For our experiments, we configure KCensus to optimize average end-to-end latency, matching the optimization objective used to configure the baselines.

Workloads. Unless otherwise specified, we focus on write requests, as all systems rely on the same well-understood read algorithm [17]. Clients issue writes to keys, with each key managed by an independent SMR shard. Our default workload is conflict-free: the key space is partitioned per client, so concurrent client requests do not conflict. This isolates the conflict-free fast path our evaluation targets. We later study workloads with conflicts.

Each write executes in $\approx 1 \mu\text{s}$, and both requests and replies are 16 B. All experiments use 100,000 keys and, unless stated otherwise, an aggregate client throughput of 1,000 req/s. Clients generate requests at random intervals following an exponential distribution, and requests are distributed uniformly across clients. Once all processes are ready, each experiment starts with a 5 s warmup, after which clients measure the latency of each request during a 10 s interval. Once a client finishes its measurements, it sustains the load on the system until all clients finish their measurements.

Our evaluation addresses the following questions:

- What is KCensus’s end-to-end latency (§7.1)?
- How do failures impact KCensus’s latency (§7.2)?
- How does KCensus’s latency scale with replicas (§7.3)?
- What is KCensus’s resource consumption (§7.4)?
- How does contention affect KCensus’s latency (§7.5)?
- How much load can KCensus sustain (§7.6)?
- How fast are optimal requirements computed (§7.7)?

7.1 End-to-end latency

We first measure the end-to-end latency of our KCensus-based key-value store and compare it with the baselines across the four 7-replica deployments.

Figure 7 reports latency per client and aggregated. In all four deployments, KCensus achieves lower average latency than the baselines. Compared to the fastest baseline in each deployment, KCensus lowers average latency by 15% in NH, 15% in EA, 12% in EU, and 9% in NA. The gain is largest in the NH and EA deployments, where the fastest baselines pay a larger latency penalty for reaching beyond the nearest majority.

KCensus also has the best 99th percentile in all deployments. In EA, the worst-located client of the best baseline has 23% higher latency than KCensus’s worst-located client, leading to a 23% higher 99th percentile. More generally, KCensus matches or beats the best baseline at every client location, reducing latency by up to 37%.

Compatibility often adds no latency beyond reaching a client’s closest majority. With $f=3$, two strategies need four intersection witnesses. If either proposer belongs to the intersection, its quorum supplies all four witnesses. Otherwise, if the intersection has size i , its replicas together with the two proposers already provide $i+2$ witnesses. Thus, a two-replica

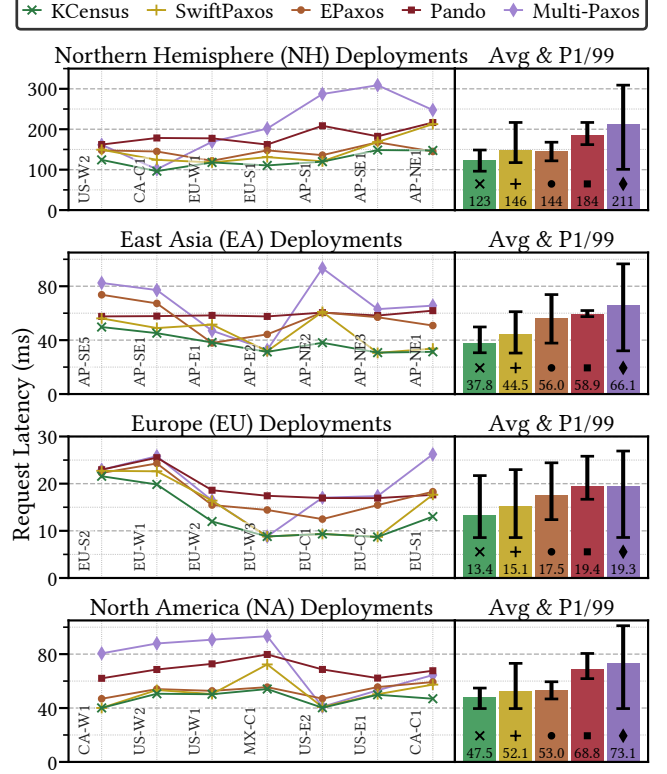


Figure 7. Latency across clients for the NH, EA, EU, and NA deployments. Bars indicate the average while the error bars indicate 1st and 99th percentiles.

intersection suffices, while a single-replica intersection needs only one additional witness.

This makes the cheapest valid strategy sufficient for every client in EU and NA: each pays only the latency to its closest majority. In EU, every pair of closest-majority quorums intersects in at least two replicas. NA proposers use three closest-majority quorums: one shared by the eastern proposers, one by the western ones, and MX-C1’s own. Only the eastern proposers’ and MX-C1’s quorums intersect in a single replica, US-E2. While waiting for their quorum’s farthest member, CA-W1, eastern proposers spread evidence of US-E2’s acceptance across the east, supplying the fourth witness without extending the fast path.

EA and NH have more single-replica intersections, so the fourth witness is not always free. Still, 5 of 7 clients in each use the cheapest strategy; the others add less than 0.3 ms to average latency in EA, and about 2 ms in NH, which lacks a hub shared by most closest-majority quorums.

Across all four deployments, only one client—AP-NE1 in NH—needs a fast-path quorum of 5 replicas rather than a bare majority. Every other client completes before the latency of reaching its fifth closest replica.

Overall, KCensus significantly lowers both average and tail latency relative to the baselines.

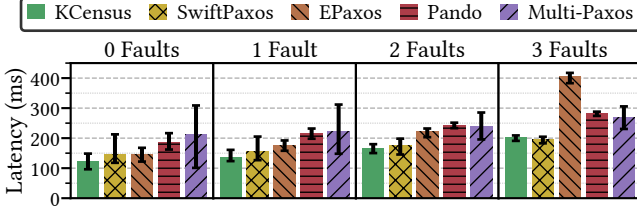


Figure 8. Average request latency in NH deployments under 0–3 replica failures. Error bars span the best- and worst-located clients’ averages across fault placements.

7.2 Impact of failures on latency

We next study whether the evaluated protocols preserve low end-to-end latency as replicas fail. We repeat Section 7.1 for every placement of 0–3 replica failures. Clients co-located with failed replicas keep issuing requests, as non-voting proposers in KCensus and SwiftPaxos, and otherwise through the replica offering them the lowest execution latency. We focus on NH, as it is the most representative deployment for global services, and report its latency averaged across clients and fault placements.

Figure 8 shows that KCensus maintains low latency as it re-synthesizes fast paths to adapt to failures. Compared to the fastest baseline at each fault count, KCensus lowers average latency by 15% with no failures, 13% with one failure, and 6% with two failures.

At three failures, the surviving configurations force all proposers onto the same quorum—the structure used by SwiftPaxos—and SwiftPaxos edges ahead of KCensus by 3%. SwiftPaxos owes this margin to speculative execution, which lets a proposer commit in a single round trip even when it is co-located with a failed replica. Because SwiftPaxos relies on a leader to order commands, it rarely has to roll back a speculative execution, making speculation cheap. KCensus does not speculate, but comes close via delegated execution (§6).

7.3 Scalability of latency with replicas

We now study how latency changes as the number of replicas grows. We re-run the experiment in Section 7.1 on deployments ranging from 3 to 31 servers. We consider two deployment series. The *Random* series starts from three random AWS regions and adds two random regions at each step. The *Parisian* series starts from Paris and repeatedly adds the two remaining regions with the lowest latency to Paris.

Figure 9 shows that KCensus consistently remains the lowest-latency configuration as the deployment grows. With three replicas, all fast-path protocols have limited placement choices and achieve similar latency. As more replicas are added, the protocols diverge: KCensus can exploit the additional placement choices, whereas the baselines remain constrained by their leader, quorum, or overlap structure.

In Random deployments, latency is not monotonic. Adding regions can both introduce more distant clients and create

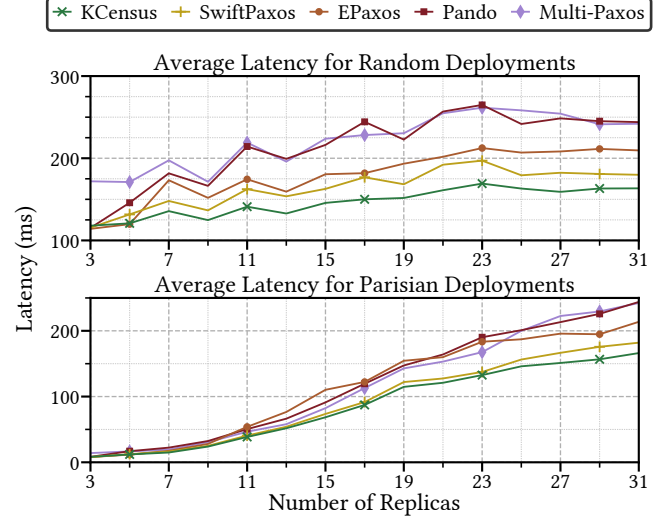


Figure 9. Average request latency for Random and Parisian deployments as the number of replicas grows from 3 to 31.

new quorum choices, so the average depends on the resulting topology. At 3 replicas, KCensus performs similarly to EPaxos, Pando, and SwiftPaxos as they are all optimal. The same applies to EPaxos at 5 replicas. With more replicas, KCensus dominates, reducing latency over the fastest baseline by 8% (at 7 replicas) to 16% (at 21).

In Parisian deployments, latency grows more smoothly because replicas are added in order of increasing distance from Paris.

Overall, latency depends on where replicas are rather than on how many there are, and KCensus synthesizes its requirements to make the most of each placement.

7.4 Resource consumption

We now quantify the cost of KCensus using the scalability experiments of Section 7.3. We measure per-replica network traffic and messages, CPU use, and memory consumption.

Bandwidth consumption. KCensus’s communication pattern is synthesized from its knowledge requirements, so its network cost does not follow a fixed quorum pattern. Figure 10 reports average bytes and messages per application request per replica in Random deployments.

KCensus’s traffic remains low as one identifier encodes knowledge (§5.4). At 31 replicas, it sends 146 B per request per replica, 40% more than Multi-Paxos but 4% less than EPaxos. Every SwiftPaxos replica acknowledges every other, making its aggregate traffic quadratic and its per-replica traffic linear; it reaches 491 B, 3.4× KCensus’s.

KCensus sends the fewest messages through 9 replicas. As its requirements add witnesses and evidence routes, KCensus reaches 4.4 per request per replica at 31 replicas—50% more than EPaxos but fewer than Pando. SwiftPaxos sends 34.1 (7.8× as many), despite grouping acknowledgements.

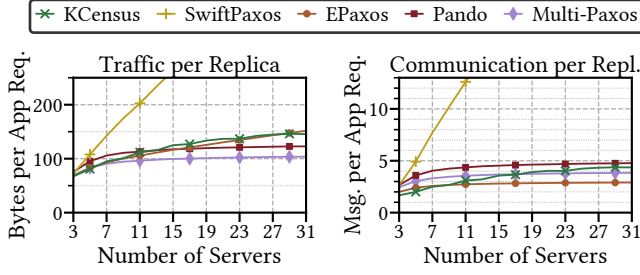


Figure 10. Average bytes (left) and messages (right) per application request per replica in Random deployments.

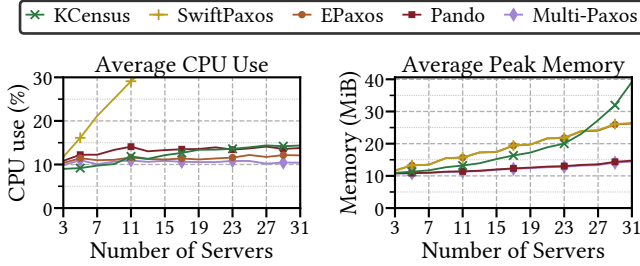


Figure 11. Average CPU use per server (left) and average peak memory per server (right) for Random deployments.

Compute consumption. The left side of Figure 11 reports the average CPU use of a server, where 100% means one core busy for the whole run, including user and kernel time. Message handling dominates, so each system’s CPU use tracks its communication cost. KCensus’s is the lowest of all systems at small scales and grows with the amount of evidence disseminated. KCensus uses 9.0% of a core at 3 replicas and 14.4% at 31, where it is 19% above EPaxos and 38% above Multi-Paxos. SwiftPaxos instead grows linearly with the number of replicas, as each server acknowledges every other, reaching 55.9% at 31 replicas, 3.9× KCensus’s use.

Memory consumption. The right side of Figure 11 reports the average peak memory of a server. KCensus uses memory to store knowledge and synthesized dissemination metadata, so its footprint grows with the number of replicas: from 10.9 MiB at 3 replicas to 39.2 MiB at 31, the largest of all systems at that scale. Multi-Paxos and Pando, which only retain the value a replica accepted, remain below 15 MiB with 31 replicas, while EPaxos and SwiftPaxos, which track a dependency set per command, sit in between at 26 MiB.

Overall, KCensus’s resource consumption remains in the range of existing fast-path protocols: at 31 replicas, it sends slightly less data per replica than EPaxos, and it is several times cheaper than SwiftPaxos on networking and CPU. What KCensus trades for lower latency is a modest message and CPU overhead over leader-based protocols, and the

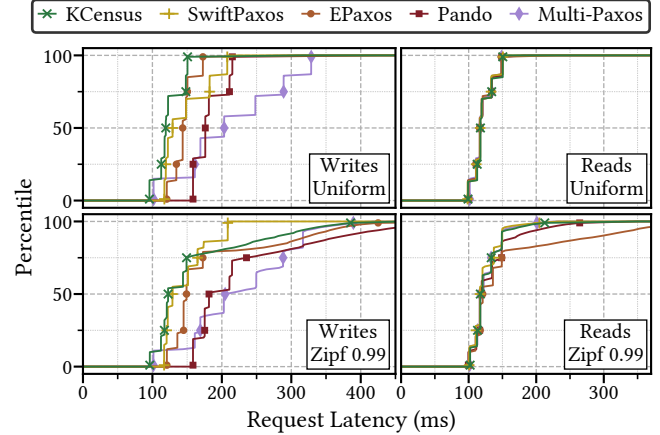


Figure 12. Latency CDFs of writes and reads in NH with a shared key space, under uniform and Zipfian access.

largest memory footprint of all systems at 31 replicas, which remains small in absolute terms—under 40 MiB per server.

7.5 Latency under contention

We measure the effect of conflicts in the NH deployment by having clients access 100,000 shared keys. Clients issue 50% reads and 50% writes at an aggregate 1,000 req/s, drawing keys either uniformly or from a Zipfian distribution with skew 0.99, matching YCSB’s workload A [21]. At this load, uniform access produces negligible conflicts and serves as a reference; Zipfian access concentrates concurrent requests on hot keys, creating many conflicts across clients.

Figure 12 reports writes and reads separately, as reads are not replicated (§6). Uniform access resembles the conflict-free result of §7.1: KCensus’s median write latency is 119 ms, and its CDF is concentrated around the fast-path latency.

Under Zipfian access, KCensus’s median is 123 ms, within 3% of the uniform median, but the 99th percentile rises to 385 ms. The CDF exposes two regimes: conflict-free writes retain the fast-path latency, while conflicting writes adopt a safe command and complete using Paxos’s second phase (§6). Averaged over writes, KCensus reaches 158 ms; SwiftPaxos is faster at 146 ms, with a shorter tail of 209 ms, because its leader-based fallback runs alongside the fast path and can order conflicting commands without recovery. Nevertheless, KCensus commits half of its writes faster than any baseline and remains ahead of EPaxos (183 ms), Multi-Paxos, and Pando (both above 220 ms).

Reads resemble uniform access, since we give every baseline KSMR’s read optimization (§6). Under Zipfian access, KCensus averages 127 ms, with a 99th percentile of 213 ms. EPaxos degrades most, to 157 ms and 433 ms, because its reads wait for every ongoing command at a majority rather than only the highest ongoing slot. SwiftPaxos avoids this: if the leader replies in the read quorum, the reader waits only for writes the leader accepted.

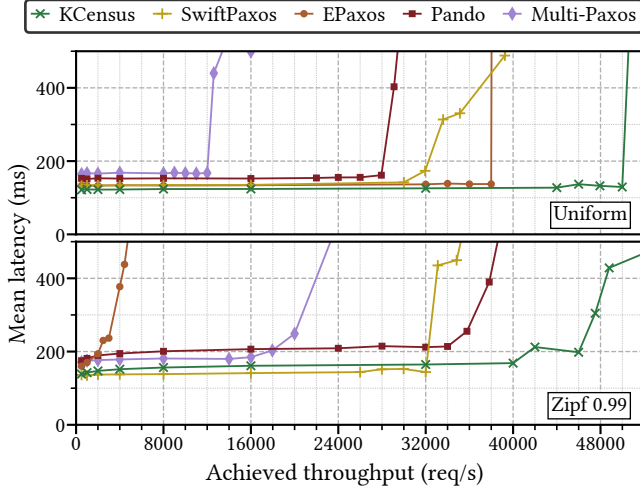


Figure 13. Mean request latency versus achieved throughput in NH under uniform and Zipfian access. We omit runs beyond saturation, where more load reduced throughput.

7.6 Latency under load

We measure how latency changes with load and how much throughput each system sustains before saturation. We reuse the workload of §7.5, increasing the request rate until each system saturates. All systems batch requests that queue up: KCensus, Pando, and Multi-Paxos group a key’s pending commands into its next slot, while EPaxos and SwiftPaxos batch as their reference implementations do, grouping commands into one instance and acknowledgements for different keys into one message, respectively. The one exception is that EPaxos never groups commands across keys, because doing so adds conflicts and does not improve its throughput in our experiments.

Figure 13 plots mean latency against achieved throughput. Under uniform access, KCensus remains near 124 ms through 44k req/s and below 137 ms up to 50k, where it saturates, giving it the lowest latency at every measured load and the highest saturation throughput. EPaxos’s latency rises sharply beyond 38k req/s, SwiftPaxos’s beyond 32k, Pando’s beyond 28k, and Multi-Paxos’s beyond 12k. KCensus, EPaxos, and SwiftPaxos saturate when their busiest replica exhausts its core; Multi-Paxos saturates earlier, bottlenecked by its leader.

Under Zipfian access, conflicts raise KCensus’s latency, but it stays below 170 ms up to 40k req/s and then degrades gradually as we batch more commands on hot keys. EPaxos degrades earliest, exceeding 370 ms by 4k req/s as conflicts grow its dependency sets. SwiftPaxos has lower latency than KCensus, as in §7.5, until it rises sharply beyond 32k req/s. Multi-Paxos and Pando rise sharply beyond 18k and 34k req/s, and remain slower than KCensus throughout.

Overall, KCensus achieves the highest throughput at comparable latency under both distributions; conflicts raise its latency but does not cause a throughput collapse.

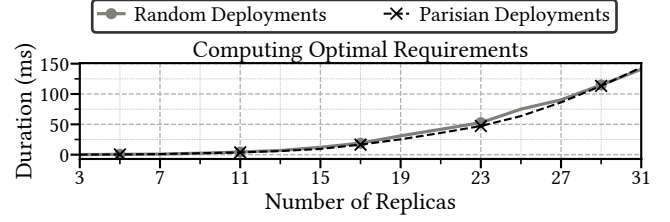


Figure 14. Time to optimize requirements on a single core.

7.7 Time to optimize requirements

Whenever KCensus processes detect a significant change in network topology or latency, they recompute optimal knowledge requirements. Due to the combinatorial complexity of the problem, one could fear that this cost would prevent KCensus from quickly adapting to network fluctuations. We evaluate this cost by running the same experiments as in Section 7.3, and measuring how long requirements optimization takes. Figure 14 reports the results.

Optimization time is similar for both kinds of deployments, ranging from 0.1 ms for 3 replicas to 144 ms for 31 replicas. This cost is orders of magnitude lower than the time interval between significant changes in topology, so it is negligible.

8 Related Work

KCensus relates to work on strongly consistent replication, low-latency consensus, topology-aware communication, and knowledge-based distributed protocols.

Strongly consistent wide-area replication. Wide-area systems face a fundamental latency–consistency tension. Many systems reduce latency by weakening consistency [3, 20, 23, 26, 27, 30, 40, 52, 59, 62–64, 74, 84, 87, 88]. Strongly consistent services instead tend to rely on consensus to order updates across regions [7, 12, 13, 22, 32, 45, 89]. Rather than relaxing consistency, KCensus reduces the latency of this consensus layer by synthesizing low-latency fast paths.

Low-latency consensus. A large body of work reduces consensus latency by fixing a particular communication pattern, quorum rule, or coordinator placement [4, 9, 10, 31, 44, 57, 65, 67–69, 85, 92]. Each of these choices is one point in the fast-path design space: they work well in some cases, but no single prior scheme is best for all topologies, workloads, and latency objectives. KCensus takes a complementary approach: it characterizes which fast paths are recoverable, and synthesizes an optimal ensemble for a given deployment.

Prior work considers fast paths in the context of quorum systems [41]. In contrast, KCensus captures a broader design space than quorum choice alone. Existing work studies fast-quorum bounds [58, 73, 79]; such bounds do not apply to KCensus, where each proposer is assigned a specific strategy.

Several protocols reduce how often proposals conflict. Some exploit commutativity [11, 56, 68, 69], while others use sequencing or timing assumptions to steer concurrent

commands apart [8, 90]. These techniques are orthogonal to KCensus: they reduce the probability of conflicting fast paths, whereas KCensus optimizes fast-path latency.

Other low-latency systems build on assumptions outside KCensus’s model. Some target Byzantine settings [33, 37, 47, 51, 82], including Heterogeneous Paxos [83], which configures whom each learner trusts and how many failures it tolerates; others exploit datacenter hardware such as RDMA or programmable switches [1, 2, 24, 25, 39, 46, 60, 75]. KCensus targets crash-fault consensus over ordinary WANs.

Topology-aware communication. Prior work showed that communication structure matters for consensus performance [18, 48–50, 66, 67, 71, 77, 85]. Ring- and tree-based protocols distribute load and improve throughput [6, 38, 48–50, 66, 67, 71, 77], while Pig Paxos [18] uses aggregation and piggybacking to reduce bandwidth bottlenecks. KCensus’s optimized dissemination graphs can resemble such structures, but are derived from a different objective: they deliver exactly the knowledge required by the synthesized fast paths along low-latency routes.

Knowledge-based protocols. Reasoning about knowledge [34, 35, 42] is a classical way to characterize the information needed to solve distributed tasks [19]. This line of work has led to optimal synchronous consensus protocols under crash, Byzantine, and omission failures [5, 14, 15, 29, 70]. KCensus shares similarities, but works in a different model: asynchronous crash-fault fast paths with non-uniform latencies. This vastly changes both the problem and the solution.

9 Conclusion

We presented KCensus, a framework that synthesizes consensus fast paths. By characterizing the evidence needed for safe recovery, KCensus turns fast-path design into a checkable optimization problem. The resulting fast paths adapt to network topology, proposer distribution, and latency objective. In our evaluation, this achieves the lowest latency across deployments, cutting average latency by up to 16%.

Future work includes mechanically checking KCensus and extending it to support EPaxos-style dependency tracking, Byzantine faults, speculative execution, and erasure coding.

Acknowledgments

We thank our shepherd and anonymous reviewers for their thoughtful feedback, and our anonymous artifact evaluators for assessing our prototype. We are also grateful to Zakaria Choukri for his work on the infrastructure supporting our AWS experiments. This work was supported in part by the Swiss National Science Foundation (SNSF) under grant 200021_215383.

References

- [1] Marcos K. Aguilera, Naama Ben-David, Rachid Guerraoui, Virendra J. Marathe, Athanasios Xygkis, and Igor Zablotchi. 2020. Microsecond Consensus for Microsecond Applications. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation* (Virtual Event) (OSDI ’20). USENIX Association, Berkeley, CA, USA, Article 34, 18 pages. <https://www.usenix.org/conference/osdi20/presentation/aguilera>
- [2] Marcos K. Aguilera, Naama Ben-David, Rachid Guerraoui, Antoine Murat, Athanasios Xygkis, and Igor Zablotchi. 2023. uBFT: Microsecond-Scale BFT Using Disaggregated Memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) (ASPLOS ’23). Association for Computing Machinery, New York, NY, USA, 862–877. doi:10.1145/3575693.3575732
- [3] Ailidani Ailijiang, Aleksey Charapko, and Murat Demirbas. 2019. Dissecting the Performance of Strongly-Consistent Replication Protocols. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) (SIGMOD ’19). Association for Computing Machinery, New York, NY, USA, 1696–1710. doi:10.1145/3299869.3319893
- [4] Ailidani Ailijiang, Aleksey Charapko, Murat Demirbas, and Tevfik Kosar. 2020. WPaxos: Wide Area Network Flexible Consensus. *IEEE Trans. Parallel Distrib. Syst.* 31, 1 (Jan. 2020), 211–223. doi:10.1109/TPDS.2019.2929793
- [5] Kaya Alpturer, Joseph Y. Halpern, and Ron van der Meyden. 2023. Optimal Eventual Byzantine Agreement Protocols with Omission Failures. In *Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing* (Orlando, FL, USA) (PODC ’23). Association for Computing Machinery, New York, NY, USA, 244–252. doi:10.1145/3583668.3594573
- [6] Y. Amir, L. E. Moser, P. M. Melliar-Smith, D. A. Agarwal, and P. Ciarfella. 1995. The Totem single-ring ordering and membership protocol. *ACM Trans. Comput. Syst.* 13, 4 (Nov. 1995), 311–342. doi:10.1145/210223.210224
- [7] Jason Baker, Chris Bond, James C. Corbett, J. J. Furman, Andrey Khorlin, James Larson, Jean-Michel Leon, Yawei Li, Alexander Lloyd, and Vadim Yushprakh. 2011. Megastore: Providing Scalable, Highly Available Storage for Interactive Services. In *Fifth Biennial Conference on Innovative Data Systems Research, CIDR 2011, Asilomar, CA, USA, January 9-12, 2011, Online Proceedings*. www.cidrdb.org, 223–234. http://cidrdb.org/cidr2011/Papers/CIDR11_Paper32.pdf
- [8] Mahesh Balakrishnan, Dahlia Malkhi, John D. Davis, Vijayan Prabhakaran, Michael Wei, and Ted Wobber. 2013. CORFU: A distributed shared log. *ACM Trans. Comput. Syst.* 31, 4, Article 10 (Dec. 2013), 24 pages. doi:10.1145/2535930
- [9] Christian Berger, Hans P. Reiser, João Sousa, and Alysso Bessani. 2019. Resilient Wide-Area Byzantine Consensus Using Adaptive Weighted Replication. In *2019 38th Symposium on Reliable Distributed Systems (SRDS)*. 183–192. doi:10.1109/SRDS47363.2019.00029
- [10] Christian Berger, Lívio Rodrigues, Hans P. Reiser, Vinicius Cogo, and Alysso Bessani. 2024. Chasing Lightspeed Consensus: Fast Wide-Area Byzantine Replication with Mercury. In *Proceedings of the 25th International Middleware Conference* (Hong Kong, Hong Kong) (Middleware ’24). Association for Computing Machinery, New York, NY, USA, 158–171. doi:10.1145/3652892.3700756
- [11] Matthew Burke, Audrey Cheng, and Wyatt Lloyd. 2020. Gryff: unifying consensus and shared registers. In *Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation* (Santa Clara, CA, USA) (NSDI ’20). USENIX Association, USA, 591–617. <https://www.usenix.org/conference/nsdi20/presentation/burke>
- [12] Mike Burrows. 2006. The Chubby lock service for loosely-coupled distributed systems. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation - Volume 7* (Seattle, WA) (OSDI ’06). USENIX Association, USA,

- 335–350. <https://www.usenix.org/conference/osdi-06/presentation/chubby-lock-service-loosely-coupled-distributed-systems>
- [13] Brad Calder, Ju Wang, Aaron Ogus, Niranjana Nilakantan, Arild Skjolsvold, Sam McKelvie, Yikang Xu, Shashwat Srivastav, Jiesheng Wu, Huseyin Simitci, Jaidev Haridas, Chakravarthy Uddaraju, Hemal Khatri, Andrew Edwards, Vaman Bedekar, Shane Mainali, Rafay Abbasi, Arpit Agarwal, Mian Fahim ul Haq, Muhammad Ikram ul Haq, Deepali Bhardwaj, Sowmya Dayanand, Anitha Adusumilli, Marvin McNett, Sriram Sankaran, Kavitha Manivannan, and Leonidas Rigas. 2011. Windows Azure Storage: a highly available cloud storage service with strong consistency. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles* (Cascais, Portugal) (SOSP '11). Association for Computing Machinery, New York, NY, USA, 143–157. doi:10.1145/2043556.2043571
 - [14] Armando Castañeda, Yannai A. Gonczarowski, and Yoram Moses. 2022. Unbeatable consensus. *Distrib. Comput.* 35, 2 (April 2022), 123–143. doi:10.1007/s00446-021-00417-3
 - [15] Armando Castañeda, Yoram Moses, Michel Raynal, and Matthieu Roy. 2017. Early Decision and Stopping in Synchronous Consensus: A Predicate-Based Guided Tour. In *Networked Systems*, Amr El Abbadi and Benoît Garbinato (Eds.). Springer International Publishing, Cham, 206–221.
 - [16] Tushar Deepak Chandra and Sam Toueg. 1996. Unreliable failure detectors for reliable distributed systems. *J. ACM* 43, 2 (March 1996), 225–267. doi:10.1145/226643.226647
 - [17] Aleksey Charapko, Ailidani Ailijiang, and Murat Demirbas. 2019. Linearizable Quorum Reads in Paxos. In *11th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 19)*. USENIX Association, Renton, WA. <https://www.usenix.org/conference/hotstorage19/presentation/charapko>
 - [18] Aleksey Charapko, Ailidani Ailijiang, and Murat Demirbas. 2021. Pig-Paxos: Devouring the Communication Bottlenecks in Distributed Consensus. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) (SIGMOD '21). Association for Computing Machinery, New York, NY, USA, 235–247. doi:10.1145/3448016.3452834
 - [19] Brian A. Coan. 1986. A communication-efficient canonical form for fault-tolerant distributed protocols. In *Proceedings of the Fifth Annual ACM Symposium on Principles of Distributed Computing* (Calgary, Alberta, Canada) (PODC '86). Association for Computing Machinery, New York, NY, USA, 63–72. doi:10.1145/10590.10596
 - [20] Brian F. Cooper, Raghu Ramakrishnan, Utkarsh Srivastava, Adam Silberstein, Philip Bohannon, Hans-Arno Jacobsen, Nick Puz, Daniel Weaver, and Ramana Yerneni. 2008. PNUTS: Yahoo!'s hosted data serving platform. *Proc. VLDB Endow.* 1, 2 (Aug. 2008), 1277–1288. doi:10.14778/1454159.1454167
 - [21] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing* (Indianapolis, Indiana, USA) (SoCC '10). Association for Computing Machinery, New York, NY, USA, 143–154. doi:10.1145/1807128.1807152
 - [22] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaure, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. 2013. Spanner: Google's Globally Distributed Database. *ACM Trans. Comput. Syst.* 31, 3, Article 8 (Aug. 2013), 22 pages. doi:10.1145/2491245
 - [23] Natacha Crooks, Youer Pu, Nancy Estrada, Trinabh Gupta, Lorenzo Alvisi, and Allen Clement. 2016. TARDiS: A Branch-and-Merge Approach To Weak Consistency. In *Proceedings of the 2016 International Conference on Management of Data* (San Francisco, California, USA) (SIGMOD '16). Association for Computing Machinery, New York, NY, USA, 1615–1628. doi:10.1145/2882903.2882951
 - [24] Huynh Tu Dang, Pietro Bressana, Han Wang, Ki Suh Lee, Noa Zilberman, Hakim Weatherspoon, Marco Canini, Fernando Pedone, and Robert Soulé. 2020. P4xos: Consensus as a Network Service. *IEEE/ACM Trans. Netw.* 28, 4 (Aug. 2020), 1726–1738. doi:10.1109/TNET.2020.2992106
 - [25] Huynh Tu Dang, Marco Canini, Fernando Pedone, and Robert Soulé. 2016. Paxos Made Switch-y. *SIGCOMM Comput. Commun. Rev.* 46, 2 (May 2016), 18–24. doi:10.1145/2935634.2935638
 - [26] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. 1987. Epidemic algorithms for replicated database maintenance. In *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing* (Vancouver, British Columbia, Canada) (PODC '87). Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/41840.41841
 - [27] Jiaqing Du, Călin Iorgulescu, Amitabha Roy, and Willy Zwaenepoel. 2014. GentleRain: Cheap and Scalable Causal Consistency with Physical Clocks. In *Proceedings of the ACM Symposium on Cloud Computing* (Seattle, WA, USA) (SOCC '14). Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/2670979.2670983
 - [28] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the presence of partial synchrony. *J. ACM* 35, 2 (April 1988), 288–323. doi:10.1145/42282.42283
 - [29] Cynthia Dwork and Yoram Moses. 1990. Knowledge and common knowledge in a Byzantine environment: Crash failures. *Information and Computation* 88, 2 (1990), 156–186. doi:10.1016/0890-5401(90)90014-9
 - [30] Mostafa Elhemali, Niall Gallagher, Nick Gordon, Joseph Idziorek, Richard Krog, Colin Lazier, Erben Mo, Akhilesh Mritunjai, Soma Sundaram Perianayagam, Tim Rath, Swami Sivasubramanian, James Christopher Sorenson III, Sroaj Sosothikul, Doug Terry, and Akshat Vig. 2022. Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 1037–1048. <https://www.usenix.org/conference/atc22/presentation/elhemali>
 - [31] Vitor Enes, Carlos Baquero, Tuanir França Rezende, Alexey Gotsman, Matthieu Perrin, and Pierre Sutra. 2020. State-machine replication for planet-scale systems. In *Proceedings of the Fifteenth European Conference on Computer Systems* (Heraklion, Greece) (EuroSys '20). Association for Computing Machinery, New York, NY, USA, Article 24, 15 pages. doi:10.1145/3342195.3387543
 - [32] etcd Authors. 2025. etcd: A Distributed, Reliable Key-Value Store for the Most Critical Data of a Distributed System. <https://etcd.io/> Accessed: 2025-03-25.
 - [33] Ittay Eyal, Adem Efe Gencer, Emin Gün Sirer, and Robbert Van Renesse. 2016. Bitcoin-NG: a scalable blockchain protocol. In *Proceedings of the 13th Usenix Conference on Networked Systems Design and Implementation* (Santa Clara, CA) (NSDI '16). USENIX Association, USA, 45–59.
 - [34] Ronald Fagin, Joseph Y. Halpern, Moshe Y. Vardi, and Yoram Moses. 1995. *Reasoning about knowledge*. MIT Press, Cambridge, MA, USA.
 - [35] Ronald Fagin, Yoram Moses, Joseph Y. Halpern, and Moshe Y. Vardi. 1995. Knowledge-based programs. In *Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing* (Ottawa, Ontario, Canada) (PODC '95). Association for Computing Machinery, New York, NY, USA, 153–163. doi:10.1145/224964.224982
 - [36] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. 1985. Impossibility of distributed consensus with one faulty process. *J. ACM* 32, 2 (April 1985), 374–382. doi:10.1145/3149.214121
 - [37] Ankit Gangwal, Haripriya Ravali Gangavalli, and Apoorva Thirupathi. 2023. A survey of Layer-two blockchain protocols. *Journal of Network*

- and Computer Applications 209 (2023), 103539. doi:10.1016/j.jnca.2022.103539
- [38] Rachid Guerraoui, Ron R. Levy, Bastian Pochon, and Vivien Quéma. 2010. Throughput optimal total order broadcast for cluster environments. *ACM Trans. Comput. Syst.* 28, 2, Article 5 (July 2010), 32 pages. doi:10.1145/1813654.1813656
- [39] Rachid Guerraoui, Antoine Murat, Javier Picorel, Athanasios Xygkis, Huabing Yan, and Pengfei Zuo. 2022. uKharon: A Membership Service for Microsecond Applications. In *Proceedings of the 2022 USENIX Annual Technical Conference* (Carlsbad, CA, USA) (USENIX ATC '22). USENIX Association, Berkeley, CA, USA, 101–120. <https://www.usenix.org/conference/atc22/presentation/guerraoui>
- [40] Rachid Guerraoui, Matej Pavlovic, and Dragos-Adrian Seredinschi. 2016. Incremental Consistency Guarantees for Replicated Objects. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*. USENIX Association, Savannah, GA, 169–184. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/guerraoui>
- [41] Rachid Guerraoui and Marko Vukolić. 2007. Refined quorum systems. In *Proceedings of the Twenty-Sixth Annual ACM Symposium on Principles of Distributed Computing* (Portland, Oregon, USA) (PODC '07). Association for Computing Machinery, New York, NY, USA, 119–128. doi:10.1145/1281100.1281120
- [42] Joseph Y. Halpern and Yoram Moses. 1990. Knowledge and common knowledge in a distributed environment. *J. ACM* 37, 3 (July 1990), 549–587. doi:10.1145/79147.79161
- [43] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: a correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.* 12, 3 (July 1990), 463–492. doi:10.1145/78969.78972
- [44] Heidi Howard, Dahlia Malkhi, and Alexander Spiegelman. 2016. Flexible Paxos: Quorum Intersection Revisited. In *20th International Conference on Principles of Distributed Systems, OPODIS 2016, Madrid, Spain, December 13–16, 2016 (LIPIcs, Vol. 70)*, Panagioti Fatourou, Ernesto Jiménez, and Fernando Pedone (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 25:1–25:14. doi:10.4230/LIPICs.OPODIS.2016.25
- [45] Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed. 2010. ZooKeeper: Wait-free Coordination for Internet-scale Systems. In *Proceedings of the 2010 USENIX Conference on USENIX Annual Technical Conference* (Boston, MA) (USENIX ATC '10). USENIX Association, USA. <https://www.usenix.org/conference/usenix-atc-10/zookeeper-wait-free-coordination-internet-scale-systems>
- [46] Sagar Jha, Jonathan Behrens, Theo Gkountouvas, Mae Milano, Weijia Song, Edward Tremel, Robbert Van Renesse, Sydney Zink, and Kenneth P. Birman. 2019. Derecho: Fast State Machine Replication for Cloud Services. *ACM Trans. Comput. Syst.* 36, 2, Article 4 (April 2019), 49 pages. doi:10.1145/3302258
- [47] Harry Kalodner, Steven Goldfeder, Xiaoqi Chen, S. Matthew Weinberg, and Edward W. Felten. 2018. Arbitrum: scalable, private smart contracts. In *Proceedings of the 27th USENIX Conference on Security Symposium* (Baltimore, MD, USA) (SEC '18). USENIX Association, USA, 1353–1370.
- [48] Eleftherios Kokoris-Kogias. 2019. Robust and Scalable Consensus for Sharded Distributed Ledgers. Cryptology ePrint Archive, Paper 2019/676. <https://eprint.iacr.org/2019/676>
- [49] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Nicolas Gailly, Ismail Khoffi, Linus Gasser, and Bryan Ford. 2016. Enhancing bitcoin security and performance with strong consistency via collective signing. In *Proceedings of the 25th USENIX Conference on Security Symposium* (Austin, TX, USA) (SEC '16). USENIX Association, USA, 279–296.
- [50] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Ford. 2018. OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding. In *2018 IEEE Symposium on Security and Privacy (SP)*. 583–598. doi:10.1109/SP.2018.000-5
- [51] Ramakrishna Kotla, Lorenzo Alvisi, Mike Dahlin, Allen Clement, and Edmund Wong. 2010. Zyzzyva: Speculative Byzantine fault tolerance. *ACM Trans. Comput. Syst.* 27, 4, Article 7 (Jan. 2010), 39 pages. doi:10.1145/1658357.1658358
- [52] Avinash Lakshman and Prashant Malik. 2010. Cassandra: a decentralized structured storage system. *SIGOPS Oper. Syst. Rev.* 44, 2 (April 2010), 35–40. doi:10.1145/1773912.1773922
- [53] Leslie Lamport. 1978. Time, Clocks, and the Ordering of Events in a Distributed System. *Commun. ACM* 21, 7 (jul 1978), 558–565. doi:10.1145/359545.359563
- [54] Leslie Lamport. 1987. A Fast Mutual Exclusion Algorithm. *ACM Transactions on Computer Systems* 5, 1 (1987), 1–11. doi:10.1145/7351.7352
- [55] Leslie Lamport. 1998. The part-time parliament. *ACM Trans. Comput. Syst.* 16, 2 (May 1998), 133–169. doi:10.1145/279227.279229
- [56] Leslie Lamport. 2005. *Generalized Consensus and Paxos*. Technical Report MSR-TR-2005-33. Microsoft Research. 60 pages. <https://www.microsoft.com/en-us/research/publication/generalized-consensus-and-paxos/>
- [57] Leslie Lamport. 2006. Fast Paxos. *Distrib. Comput.* 19, 2 (Oct. 2006), 79–103. doi:10.1007/s00446-006-0005-x
- [58] Leslie Lamport. 2006. Lower bounds for asynchronous consensus. *Distributed Computing* 19, 2 (2006), 104–125. doi:10.1007/s00446-006-0155-x
- [59] Cheng Li, Daniel Porto, Allen Clement, Johannes Gehrke, Nuno Preguiça, and Rodrigo Rodrigues. 2012. Making Geo-Replicated Systems Fast as Possible, Consistent when Necessary. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI '12)*. USENIX Association, Hollywood, CA, 265–278. <https://www.usenix.org/conference/osdi12/technical-sessions/presentation/li>
- [60] Jialin Li, Ellis Michael, Naveen Kr. Sharma, Adriana Szekeres, and Dan R. K. Ports. 2016. Just Say NO to Paxos Overhead: Replacing Consensus with Network Ordering. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*. USENIX Association, Savannah, GA, 467–483. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/li>
- [61] Greg Linden. 2006. Make Data Useful. Guest lecture, Stanford University Data Mining class. <https://glinden.blogspot.com/2006/12/slides-from-my-talk-at-stanford.html>
- [62] Wyatt Lloyd, Michael J. Freedman, Michael Kaminsky, and David G. Andersen. 2011. Don't settle for eventual: scalable causal consistency for wide-area storage with COPS. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles* (Cascais, Portugal) (SOSP '11). Association for Computing Machinery, New York, NY, USA, 401–416. doi:10.1145/2043556.2043593
- [63] Wyatt Lloyd, Michael J. Freedman, Michael Kaminsky, and David G. Andersen. 2013. Stronger semantics for low-latency geo-replicated storage. In *Proceedings of the 10th USENIX Conference on Networked Systems Design and Implementation* (Lombard, IL) (NSDI '13). USENIX Association, USA, 313–328.
- [64] Haonan Lu, Kaushik Veeraraghavan, Philippe Ajoux, Jim Hunt, Yee Jiun Song, Wendy Tobagus, Sanjeev Kumar, and Wyatt Lloyd. 2015. Existential consistency: measuring and understanding consistency at Facebook. In *Proceedings of the 25th Symposium on Operating Systems Principles* (Monterey, California) (SOSP '15). Association for Computing Machinery, New York, NY, USA, 295–310. doi:10.1145/2815400.2815426
- [65] Yanhua Mao, Flavio P. Junqueira, and Keith Marzullo. 2008. Mencius: Building Efficient Replicated State Machines for WANs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation* (San Diego, CA, USA) (OSDI '08). USENIX Association, Berkeley, CA, USA, 369–384. <https://www.usenix.org/conference/osdi-08/mencius-building-efficient-replicated-state-machines-wans>

- [66] Parisa Jalili Marandi, Marco Primi, and Fernando Pedone. 2012. Multi-Ring Paxos. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2012)*. 1–12. doi:10.1109/DSN.2012.6263916
- [67] Parisa Jalili Marandi, Marco Primi, Nicolas Schiper, and Fernando Pedone. 2010. Ring Paxos: A high-throughput atomic broadcast protocol. In *2010 IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 527–536. doi:10.1109/DSN.2010.5544272
- [68] Iulian Moraru, David G. Andersen, and Michael Kaminsky. 2013. There is more consensus in Egalitarian parliaments. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (Farmington, Pennsylvania) (SOSP '13)*. Association for Computing Machinery, New York, NY, USA, 358–372. doi:10.1145/2517349.2517350
- [69] Shuai Mu, Lamont Nelson, Wyatt Lloyd, and Jinyang Li. 2016. Consolidating Concurrency Control and Consensus for Commits under Conflicts. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*. USENIX Association, Savannah, GA, 517–532. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/mu>
- [70] Gil Neiger and Mark R. Tuttle. 1993. Common knowledge and consistent simultaneous coordination. *Distrib. Comput.* 6, 3 (April 1993), 181–192. doi:10.1007/BF02242706
- [71] Ray Neiheiser, Miguel Matos, and Luís Rodrigues. 2021. Kauri: Scalable BFT Consensus with Pipelined Tree-Based Dissemination and Aggregation. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (Virtual Event, Germany) (SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 35–48. doi:10.1145/3477132.3483584
- [72] Erik Nygren, Ramesh K. Sitaraman, and Jennifer Sun. 2010. The Akamai network: a platform for high-performance internet applications. *SIGOPS Oper. Syst. Rev.* 44, 3 (Aug. 2010), 2–19. doi:10.1145/1842733.1842736
- [73] Fernando Pedone and André Schiper. 1999. Generic Broadcast. In *Distributed Computing, 13th International Symposium (DISC 1999) (Lecture Notes in Computer Science)*. Springer, 94–106. doi:10.1007/3-540-48169-9_7
- [74] Karin Petersen, Mike J. Spreitzer, Douglas B. Terry, Marvin M. Theimer, and Alan J. Demers. 1997. Flexible update propagation for weakly consistent replication. In *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles (Saint Malo, France) (SOSP '97)*. Association for Computing Machinery, New York, NY, USA, 288–301. doi:10.1145/268998.266711
- [75] Marius Poke and Torsten Hoefer. 2015. DARE: High-Performance State Machine Replication on RDMA Networks. In *Proceedings of the 24th International Symposium on High-Performance Parallel and Distributed Computing (Portland, Oregon, USA) (HPDC '15)*. Association for Computing Machinery, New York, NY, USA, 107–118. doi:10.1145/2749246.2749267
- [76] Tuanir França Rezende and Pierre Sutra. 2020. Leaderless State-Machine Replication: Specification, Properties, Limits. In *34th International Symposium on Distributed Computing, DISC 2020 (LIPIcs, Vol. 179)*, Hagit Attiya (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 24:1–24:17. doi:10.4230/LIPIcs.DISC.2020.24
- [77] Sajjad Rizvi, Bernard Wong, and Srinivasan Keshav. 2017. Canopus: A Scalable and Massively Parallel Consensus Protocol. In *Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies (Incheon, Republic of Korea) (CoNEXT '17)*. Association for Computing Machinery, New York, NY, USA, 426–438. doi:10.1145/3143361.3143394
- [78] Fedor Ryabinin, Alexey Gotsman, and Pierre Sutra. 2024. Swift-Paxos: Fast Geo-Replicated State Machines. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, Santa Clara, CA, 345–369. <https://www.usenix.org/conference/nsdi24/presentation/ryabinin>
- [79] Fedor Ryabinin, Alexey Gotsman, and Pierre Sutra. 2025. Brief Announcement: Revisiting Lower Bounds for Two-Step Consensus. In *Proceedings of the 2025 ACM Symposium on Principles of Distributed Computing (PODC '25)*. Association for Computing Machinery, 58–61. doi:10.1145/3732772.3733541
- [80] Fedor Ryabinin, Alexey Gotsman, and Pierre Sutra. 2026. Making Democracy Work: Fixing and Simplifying Egalitarian Paxos. In *29th International Conference on Principles of Distributed Systems (OPODIS 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 361)*, Andrei Arusoae, Emanuel Onica, Michael Spear, and Sara Tucci-Piergiovanni (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 22:1–22:19. doi:10.4230/LIPIcs.OPODIS.2025.22
- [81] Fred B. Schneider. 1990. Implementing fault-tolerant services using the state machine approach: a tutorial. *ACM Comput. Surv.* 22, 4 (Dec. 1990), 299–319. doi:10.1145/98163.98167
- [82] Cosimo Sguanci, Roberto Spatafora, and Andrea Mario Vergani. 2021. Layer 2 Blockchain Scaling: a Survey. *CoRR abs/2107.10881* (2021). arXiv:2107.10881 <https://arxiv.org/abs/2107.10881>
- [83] Isaac Sheff, Xinwen Wang, Robbert van Renesse, and Andrew C. Myers. 2021. Heterogeneous Paxos. In *24th International Conference on Principles of Distributed Systems (OPODIS 2020) (LIPIcs, Vol. 184)*, Quentin Bramas, Rotem Oshman, and Paolo Romano (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 5:1–5:17. doi:10.4230/LIPIcs.OPODIS.2020.5
- [84] Atul Singh, Pedro Fonseca, Petr Kuznetsov, Rodrigo Rodrigues, and Petros Maniatis. 2009. Zeno: eventually consistent Byzantine-fault tolerance. In *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (Boston, Massachusetts) (NSDI '09)*. USENIX Association, USA, 169–184.
- [85] Joao Sousa and Alysson Bessani. 2015. Separating the WHEAT from the Chaff: An Empirical Design for Geo-Replicated State Machines. In *Proceedings of the 2015 IEEE 34th Symposium on Reliable Distributed Systems (SRDS) (SRDS '15)*. IEEE Computer Society, USA, 146–155. doi:10.1109/SRDS.2015.40
- [86] Pierre Sutra. 2020. On the correctness of Egalitarian Paxos. *Inf. Process. Lett.* 156, C (April 2020), 6 pages. doi:10.1016/j.ipl.2019.105901
- [87] Douglas B. Terry, Vijayan Prabhakaran, Ramakrishna Kotla, Mahesh Balakrishnan, Marcos K. Aguilera, and Hussam Abu-Libdeh. 2013. Consistency-based service level agreements for cloud storage. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (Farmington, Pennsylvania) (SOSP '13)*. Association for Computing Machinery, New York, NY, USA, 309–324. doi:10.1145/2517349.2522731
- [88] D. B. Terry, M. M. Theimer, Karin Petersen, A. J. Demers, M. J. Spreitzer, and C. H. Hauser. 1995. Managing update conflicts in Bayou, a weakly connected replicated storage system. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (Copper Mountain, Colorado, USA) (SOSP '95)*. Association for Computing Machinery, New York, NY, USA, 172–182. doi:10.1145/224056.224070
- [89] The Apache Software Foundation. 2025. Guarantees | Apache Cassandra Documentation. <https://cassandra.apache.org/doc/5.0/cassandra/architecture/guarantees.html>. Accessed 28-03-2025.
- [90] Sarah Tollman, Seo Jin Park, and John Ousterhout. 2021. EPaxos Revisited. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, 613–632. <https://www.usenix.org/conference/nsdi21/presentation/tollman>
- [91] Muhammed Uluyol, Anthony Huang, Ayush Goel, Mosharaf Chowdhury, and Harsha V. Madhyastha. 2020. Near-Optimal Latency Versus Cost Tradeoffs in Geo-Distributed Storage. In *Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation (Santa Clara, CA, USA) (NSDI'20)*. USENIX Association, USA, 157–180. <https://www.usenix.org/conference/nsdi20/presentation/uluyol>

- [92] Robbert Van Renesse and Deniz Altinbuken. 2015. Paxos Made Moderately Complex. *ACM Comput. Surv.* 47, 3, Article 42 (Feb. 2015), 36 pages. doi:10.1145/2673577
- [93] Jiong Yang, Gil Neiger, and Eli Gafni. 1998. Structured Derivations of Consensus Algorithms for Failure Detectors. In *Proceedings of the Seventeenth Annual ACM Symposium on Principles of Distributed Computing, PODC '98, Puerto Vallarta, Mexico, June 28 - July 2, 1998*, Brian A. Coan and Yehuda Afek (Eds.). ACM, 297–306. doi:10.1145/277697.277755

A KCensus Correctness

Recall the properties of the adopt-commit variant which we consider (presented in Section 2.2):

Termination: Propose adopts or commits if called at a correct process.

Validity: Propose adopts or commits only proposed values.

Agreement: if Propose commits v at some process, then Propose never adopts or commits $v' \neq v$ at other processes.

We show in Section A.1 that these properties are satisfied by our adopt-commit protocol presented in Section 4.

A consensus protocol should satisfy the following properties:

Termination: Propose decides if called at a correct process.

Validity: Propose decides only proposed values.

Agreement: a single value may be decided by Propose calls.

As mentioned in Section 2.2, consensus may be solved using the adopt-commit abstraction in the fast path. Algorithm 7 formalizes the construction sketched in Section 2.2, showing how adopt-commit protocols can be used to implement consensus. In Section A.2 we prove that the protocol in Algorithm 7 satisfies the three required consensus properties.

Algorithm 7. Consensus Using Adopt-Commit

```

1 def Consensus::Propose(v):
2   call AdoptCommit::Propose(v)
3 upon event AdoptCommit::Commit(v):
4   trigger Consensus::Decide(v)
5 upon event AdoptCommit::Adopt(v):
6   call FallbackConsensus::Propose(v)
7 upon event FallbackConsensus::Decide(v):
8   trigger Consensus::Decide(v)

```

A.1 KCensus's Adopt-Commit Correctness

In this section we prove the correctness of our adopt-commit protocol presented in Section 4, which serves as the core building block of KCensus. Specifically, we prove that our adopt-commit protocol satisfies the properties presented in the beginning of this section. We first clarify some assumptions. We assume a network model with perfect links. This implies that any message sent by a correct process to a correct process is guaranteed to be eventually received at the target. We further assume that any sending of messages executed throughout the protocol is done in the background and the calling block immediately continues to the next step. Additionally, we assume that any block within a method call or a message-handling routine is executed atomically, unless it contains wait steps. That is, such a block is not interrupted by input events (i.e., a call to Propose or incoming messages). In particular, this implies that `can_commit`, which contains no wait steps, executes without any incoming messages being processed at the process during its execution. As a result, the value of `known_acceptors` remains unchanged throughout.

We next establish several supporting claims to facilitate the correctness proof of our adopt-commit protocol. We start with some statements concerning the behavior of the gossip dissemination process. The subsequent observation ensures that `Accept&Spread` messages disseminate proposed values.

Observation A.1. *Only proposed values are being sent in `Accept&Spread` messages.*

Proof. `Accept&Spread` messages are sent on two occasions. They are initiated on line 10 and propagated on line 33. We prove the lemma by induction. For the base case, an `Accept&Spread` message is initiated on line 10 with value v during a `Propose(v)` call. Hence, such an `Accept&Spread` message includes a proposed value. Now suppose the lemma holds for all `Accept&Spread` messages sent so far, and consider an `Accept&Spread` message sent on line 33. It is sent with a value from a received `Accept&Spread` message, which is a proposed value by the inductive hypothesis. $\square$

The following observation and lemma ensure the dissemination of `Accept&Spread` messages when no process is frozen by some adopter.

Observation A.2. *Upon receiving an `Accept&Spread` message for the first time, a correct process sends an `Accept&Spread` message to all other processes (on line 33) containing the value from the received message, unless the process has previously received a `Freeze` message.*

Proof. Before receiving the first `Accept&Spread` message, since the handler of such messages is the only place where `known_acceptors` is modified, its values are still $\emptyset$. Let v be the value in the first received `Accept&Spread` message at a correct process p (in

case p ever receives such a message). Then upon its receipt, if p has not previously received a Freeze message, p modifies `known_acceptors[p]` on line 29 from $\emptyset$ to a value that includes p . As the value of `known_acceptors[p]` grew, p next sends to others an Accept&Spread message containing v on line 33. $\square$

Lemma A.3. *If some correct process calls Propose and does not receive any Freeze message, then every correct process eventually receives an Accept&Spread message.*

Proof. Let p be a correct process that calls Propose. During its Propose call, p sends itself an Accept&Spread message on line 10. If p did not receive any Freeze message, then by Observation A.2, p sends to others an Accept&Spread message (on line 33) following the receipt of this message if it has not done so earlier. By our network model, which assumes perfect links, every correct process eventually receives an Accept&Spread message sent by p . $\square$

The following observations pertain to accepted values. They establish that each process accepts at most one value, and that every process appearing in the values of the `known_acceptors` array at any process has accepted the same value as that process.

Observation A.4. *The value of `accepted` is assigned only non- $\perp$ values, and it is modified at most once on each process from the initial $\perp$ value to another value.*

Proof. The only assignment to `accepted` (other than its initialization) is done on line 24. On the first receipt of an Accept&Spread message, the value in this message, which is a proposed value and not $\perp$ by Observation A.1, is assigned to `accepted`. From this point on, the value of `accepted` is not $\perp$ so it remains the same. $\square$

Observation A.5. *All processes appearing in the values of the `known_acceptors` array at process p , accepted the same value as p did, i.e., assigned it to their `accepted` local variable.*

Proof. We proceed by induction. The base case vacuously holds since the `known_acceptors` values are initially empty (line 21). Now suppose the observation holds for all `known_acceptors` values at a certain point in the execution, and consider the first following update of `known_acceptors` at some process p on line 29 or line 31 upon handling an Accept&Spread(v , `knowledge_of_q`) message from some process q . p reaches this line after having assigned a value to `accepted` on line 24. Since p skipped the `if` clause on lines 25–26, `accepted` at p equals v . This is the only value p accepts by Observation A.4. The only processes added to `known_acceptors` values on lines 29 and 31 are p itself, and the processes in the values of the `knowledge_of_q` array from the Accept&Spread message received from q . `knowledge_of_q` was the value of `known_acceptors` at q when it sent the Accept&Spread message. By the inductive hypothesis, its values include processes that assigned the same value as q to their `accepted` local variable, which is the value q sent in its Accept&Spread message—namely, v . $\square$

Next, we present a couple of insights about knowledge expansion and its consequences.

Observation A.6 (knowledge does not decrease). *Processes are never removed from values of the `known_acceptors` array at any process.*

Proof. The `known_acceptors` values are updated only on lines 29 and 31, where they are modified by taking the union with the current value, thus either remaining the same or gaining additional processes. $\square$

Corollary A.7. *For any process p , the value of `known_acceptors[p]` at some process r at any point in the execution is a subset of the final value of `known_acceptors[p]` at p .*

Proof. For $p = r$, QED follows from Observation A.6. We proceed by induction for the remaining case, showing that for any processes p and r such that $p \neq r$, the value of `known_acceptors[p]` at r at any point in the execution is a subset of the final value of `known_acceptors[p]` at p . The base case vacuously holds since the `known_acceptors` values are initially empty (line 21). Now suppose the observation holds for all `known_acceptors` values at a certain point in the execution, and consider the first following update of `known_acceptors[p]` at some process r on line 31 such that $p \neq r$. It is updated to the union of its current value and the value of `knowledge_of_q[p]` (where `knowledge_of_q` is received in an Accept&Spread message from process q). By the inductive hypothesis, the value of `known_acceptors[p]` before this update is a subset of the final value of `known_acceptors[p]` at p . So it remains to prove that the value of `knowledge_of_q[p]` is also a subset of the final value of `known_acceptors[p]` at p . `knowledge_of_q[p]` was the value of `known_acceptors[p]` at q when it sent its Accept&Spread message to r . If $p = q$, then as mentioned at the beginning of this proof, `known_acceptors[p]` at p is a subset of its final value. Else, by the inductive hypothesis, `known_acceptors[p]` at q when q sent its Accept&Spread message to r is a subset of the final value of `known_acceptors[p]` at p . $\square$

We now consider the local state of processes in case some process commits.

Lemma A.8. *If process p commits, every process $q \in \text{REQUIREMENTS}[p].\text{keys}()$ appears in $\text{known_acceptors}[q]$ at p .*

Proof. Since can_commit returns True, lines 43–45 ensure that $\text{REQUIREMENTS}[p][q] \subseteq \text{known_acceptors}[q]$ at p , and the normal form gives $q \in \text{REQUIREMENTS}[p][q]$. $\square$

Corollary A.9 (Consequence of Lemma A.8 and Observation A.5). *If a process p commits value v , then every process in $\text{REQUIREMENTS}[p].\text{keys}()$ accepted v , i.e., assigned v to its accepted local variable.*

Proof. Let $q \in \text{REQUIREMENTS}[p].\text{keys}()$ for a process p that commits. By Lemma A.8, $q \in \text{known_acceptors}[q]$ at p . By Observation A.5, q accepted the same value as p . $\square$

The following observation identifies compatible requirements.

Observation A.10. *The processes in the keys of two compatible requirement dictionaries intersect.*

Proof. Consider two compatible requirement dictionaries req_a and req_b . Since they are compatible, the condition $|\text{witnesses_a} \cup \text{witnesses_b}| > f$ on line 8 evaluates to True. Hence, it is impossible that both witnesses_a and witnesses_b are empty. By their definitions on lines 6 and 7, this implies that $\text{req}_a.\text{keys}() \cap \text{req}_b.\text{keys}() \neq \emptyset$. $\square$

Next, we identify behaviors related to adopting processes.

Lemma A.11. *For any distinct proposed values v and v' , an adopter receives a Frozen reply from some process required by v or v' to know about a process in the intersection of the keys in the requirement dictionaries of v and v' .*

Proof. Let v and v' be distinct proposed values. An adopter A waits for Frozen replies from $n - f$ processes (line 16). As at most f processes are faulty, the adopter is guaranteed to receive $n - f$ replies. By definition of compatible requirements (line 3), at least $f + 1$ processes are required by v or v' to know about some process in the intersection of the keys in the requirement dictionaries of v and v' . These $f + 1$ processes intersect with the $n - f$ processes whose replies are received by adopter A . QED follows. $\square$

Lemma A.12. *If value v is committed, then any adopter invoking $\text{could_potentially_commit}$ with a proposed value $v' \neq v$ on some iteration of line 53 will obtain False.*

Proof. Consider a process r that commits value v whose associated requirement is req_v , a process r' proposing value $v' \neq v$ whose associated requirement is $\text{req}_{v'}$, and an adopter A that invokes $\text{could_potentially_commit}$ with v' on some iteration of line 53. By Lemma A.11, there exists process $q \in \text{req}_v.\text{keys}() \cap \text{req}_{v'}.\text{keys}()$, such that A receives a Frozen reply from some process p satisfying (1) $p \in \text{req}_v.\text{keys}()$ and $q \in \text{req}_v[p]$, or (2) $p \in \text{req}_{v'}.\text{keys}()$ and $q \in \text{req}_{v'}[p]$ (or both (1) and (2)).

First assume (1) holds. In this case, intuitively, since r commits, the requirement req_v is met, and in particular p knows about q accepting v ; A learns about it from p , and concludes that q did not accept v' , hence $\text{req}_{v'}$ is not met and thus v' may not be committed.

Formally, since r commits, then a call by r to $\text{can_commit}(v)$ on line 11 returns True. This means that when committing, $\text{req}_v[p] \subseteq \text{known_acceptors}[p]$ at r (lines 44–45), which implies $q \in \text{known_acceptors}[p]$ at r . By Corollary A.7, the value of $\text{known_acceptors}[p]$ at r when it commits is a subset of the final value of $\text{known_acceptors}[p]$ at p . When p handles the Freeze message from A and replies with a Frozen message, p 's known_acceptors has reached its final value, since it is modified only upon Accept&Spread messages handled before the Freeze message is handled. Consequently, $q \in \text{known_acceptors}[p]$ at p when it sends the Frozen reply to A . When A executes $\text{could_potentially_commit}$ with v' , if it does not return False before iterating over p 's reply on lines 58–60, it returns False on that iteration since p accepted $v \neq v'$ (by Corollary A.9) and $q \in \text{acceptors} \cap \text{req}_{v'}.\text{keys}()$.

Now assume (2) holds. In this case, intuitively, if p did not accept v' , then A learns about it, and concludes that $\text{req}_{v'}$ is not met and thus v' may not be committed. Else (p accepted v'), since r commits, the requirement req_v is met, and in particular q accepts v , hence p does not possess knowledge that q accepted v' . A learns about it from p , and concludes that $\text{req}_{v'}$ is not met and thus v' may not be committed.

Formally, assume first that p did not accept v' . When A executes $\text{could_potentially_commit}$ with v' , if it does not return False before iterating over p 's reply on lines 58–60, it returns False on that iteration since $p \in \text{req}_{v'}.\text{keys}()$. Now assume p accepted v' . Since r commits v , then q accepted v by Corollary A.9. By Observation A.4, q does not accept v' . By Observation A.5, p does not add q to its $\text{known_acceptors}[p]$. Therefore, when A executes $\text{could_potentially_commit}$ with v' , if it does not return False before iterating over p 's reply on lines 61–63, it returns False on that iteration since $p \in \text{req}_{v'}.\text{keys}()$, and $q \in \text{req}_{v'}[p]$ but $q \notin \text{acceptors}$. $\square$

Lemma A.13. *If value v is committed, then any adopter receives a Frozen reply with value v .*

Proof. Consider an adopter A receiving $n - f$ Frozen replies on line 16. As we assume the requirement of each process is valid, the requirement dictionary of v 's proposer includes at least $f + 1$ processes as keys. Hence, at least one of the Frozen replies received by adopter A must be from a process p in the keys of this requirement dictionary. By Corollary A.9, p accepted v . It assigned v to accepted on line 24 before receiving the Freeze message, after which p stops handling Accept&Spread messages. Hence, upon receiving a Freeze message from A , p replied with a Frozen message with value v . $\square$

Lemma A.14. *If value v is committed, then calling `could_potentially_commit` with v on line 53 by an adopter returns True.*

Proof. Consider a process r that commits value v whose associated requirement is req_v , and an adopter A that invokes `could_potentially_commit` with v on some iteration of line 53. Further consider a `Frozen( $p, val, acceptors$ )` reply from process p received by adopter A . We need to show that `could_potentially_commit` (line 56) with v does not return False during the iteration of the for loop in which this reply is processed within lines 58–63.

First assume that $p \in req_v.keys()$. By Corollary A.9, p accepted v . The acceptance must happen before sending a Frozen reply to A (when handling the incoming Freeze message), because the value of `accepted` may change only before receiving a Freeze message. Hence, val equals v . Thus, when executing `could_potentially_commit` with v , A iterates over the considered Frozen reply on lines 61–63. Since r commits, then a call by r to `can_commit( $v$ )` on line 11 returns True. This means that when committing, $req_v[p] \subseteq known_acceptors[p]$ at r (lines 43–45). By Corollary A.7, the value of `known_acceptors[p]` at r when it commits is a subset of the final value of `known_acceptors[p]` at p . When p handles the Freeze message from A and replies with a Frozen message, p 's `known_acceptors` has reached its final value, since it is modified only upon Accept&Spread messages handled before the Freeze message is handled. Consequently, $req_v[p] \subseteq known_acceptors[p]$ at p when it sends the Frozen reply to A . Thus, the condition on line 62 evaluates to False, and False is not returned on line 63 during the processing of p 's reply.

Otherwise ($p \notin req_v.keys()$), if $val = v$ then the condition on line 62 clearly evaluates to False. Else, $val \neq v$. Then when executing `could_potentially_commit` with v , A iterates over this Frozen reply on lines 58–60. By Observation A.5, all processes in `acceptors` (which is the value of `known_acceptors[p]` at process p when it sends A the Frozen reply) accepted the same value as p did, namely, val . Due to Observation A.4, these processes cannot accept another value, specifically not v . By Corollary A.9, this implies that none of these processes is in $req_v.keys()$. Hence, the condition on line 59 evaluates to False, and False is not returned on line 60 during the processing of p 's reply. $\square$

Next, we establish situations in which calling `can_commit` returns True.

Lemma A.15. *After reaching a configuration from which calling `can_commit( $v$ )` would return True at process r , a call to `can_commit( $v$ )` at r from any subsequent configuration will return True as well.*

Proof. Consider a configuration C from which calling `can_commit( $v$ )` would return True at process r . We will show that on calling `can_commit( $v$ )` from any subsequent configuration, False may not be returned on lines 42 and 45.

First, observe that `accepted = v` at configuration C , due to line 42. By Observation A.4, this holds also at any subsequent configuration. Additionally, considering lines 43–45, for any process p in the constant set `REQUIREMENTS[r].keys()`, the condition `REQUIREMENTS[r][p]  $\subseteq$  known_acceptors[p]` holds at configuration C . This holds also at any subsequent configuration, since `REQUIREMENTS` is constant, and values of `known_acceptors` may only grow due to Observation A.6. Overall, during a call to `can_commit( $v$ )` from any configuration subsequent to C , the conditions on lines 42 and 44 are not satisfied, hence the call returns True. $\square$

Claim A.16. *If all correct processes receive Accept&Spread messages such that they all receive the same value v in their first received Accept&Spread message, and none of them receives any Freeze message, then a correct process that calls `Propose( $v$ )` eventually reaches a configuration from which calling `can_commit( $v$ )` would return True.*

Proof. Let the value that appears in the first received Accept&Spread message of all correct processes be v . Let C denote the set of correct processes in the system. Assume none of them receives any Freeze message, thus they all handle any received Accept&Spread message. Let r be a correct process that calls `Propose( $v$ )`.

Assume some process in `REQUIREMENTS[r].keys()` is faulty. r 's failure detector is guaranteed to detect it at some point and send a `DoAdopt` message to r on line 50. Hence, r is guaranteed to break from the wait on line 11. But r cannot break from the wait due to receiving a `DoAdopt` message, since otherwise r would send a Freeze message to all on line 15, contradicting our assumption that no correct process receives a Freeze message. Therefore, the configuration at which r breaks from the wait is one from which calling `can_commit( $v$ )` returns True, and we are done.

Now assume all processes in `REQUIREMENTS[r].keys()` are correct. For each process $p \in C$, Observation A.2 implies that after p receives an Accept&Spread message for the first time, it sends `Accept&Spread( $v, \mathcal{K}$ )` to others on line 33, where $\mathcal{K}$ is an

array with one set of processes per process. We have $p \in \mathcal{K}[p]$ because p is explicitly added to $\text{known_acceptors}[p]$ at p on line 29. As a result, every process $q \in C$ eventually receives from every process $p \in C$ an $\text{Accept\&Spread}(v, \mathcal{K})$ message such that $p \in \mathcal{K}[p]$, and adds p to $\text{known_acceptors}[q]$ at q on line 29. Consequently, at one of its executions of line 29, q assigns to $\text{known_acceptors}[q]$ at q a superset of C for the first time. This implies that for $q = r$, $\text{known_acceptors}[q]$ at r eventually contains a superset of C . As for $q \neq r$, q sends to others on line 33 an $\text{Accept\&Spread}$ message with a known_acceptors value in which $\text{known_acceptors}[q]$ is a superset of C . Therefore, for each process $q \neq r$ in $\text{REQUIREMENTS}[r].\text{keys}()$ (which is assumed to be a correct process), r eventually receives from q an $\text{Accept\&Spread}(v, \mathcal{K}_q)$ where $C \subseteq \mathcal{K}_q[q]$, and assigns to $\text{known_acceptors}[q]$ at r a superset of C on line 31. Because the normal form gives $\text{REQUIREMENTS}[r][q] \subseteq \text{REQUIREMENTS}[r].\text{keys}()$ and every process in this key set is correct, $\text{REQUIREMENTS}[r][q] \subseteq C$. Hence, r eventually reaches a configuration in which $\text{REQUIREMENTS}[r][q] \subseteq \text{known_acceptors}[q]$ for every key q . As values of known_acceptors may only grow due to Observation A.6, r reaches a configuration from which for every process $q \in \text{REQUIREMENTS}[r].\text{keys}()$, $\text{REQUIREMENTS}[r][q] \subseteq \text{known_acceptors}[q]$ at r , hence calling $\text{can_commit}(v)$ from this configuration would return True. $\square$

We next address the termination of Propose calls in certain scenarios.

Observation A.17. *If a correct process reaches the wait on line 16 while executing a Propose call, then this wait eventually ends.*

Proof. Let r be a correct process running a call to Propose and reaching line 16. Since r is correct, given our system model, which ensures at most f faulty processes and assumes perfect links, the messages sent by r on line 15 are guaranteed to eventually reach $n - f$ correct processes. These processes will then reply, and their replies will eventually reach r , allowing the wait on line 16 to eventually complete. $\square$

Observation A.18. *Assume there exists a configuration in a given execution from which calling $\text{can_commit}(v)$ at a correct process r would return True. Then if r calls $\text{Propose}(v)$ at any point in the execution, this call is guaranteed to eventually return.*

Proof. Assume a correct process r calls $\text{Propose}(v)$ at some point. Recall our assumption that any block within a method call is executed atomically unless it contains wait steps. So it suffices to show that the wait steps performed during a Propose call (on lines 11 and 16) eventually return. If r reaches the wait on line 16, it is guaranteed to end this wait by Observation A.17. As for line 11, if the first configuration from which calling $\text{can_commit}(v)$ at r would return True occurs after r passes line 11 then we are done. Otherwise, by Lemma A.15, $\text{can_commit}(v)$ returns True at some point when r executes line 11, thus it ends the wait. $\square$

Claim A.19. *Assume a correct process r receives a DoAdopt message in a certain execution. Then if r calls Propose at any point in the execution, this call is guaranteed to eventually return.*

Proof. Assume a correct process r calls Propose at some point in a certain execution, and receives a DoAdopt message at some point in that execution. Recall our assumption that any block within a method call is executed atomically unless it contains wait steps. So it suffices to show that the wait steps performed during a Propose call (on lines 11 and 16) eventually end. The wait on line 11 is guaranteed to end due to the receipt of a DoAdopt message. As for line 16, if r reaches this wait, it is guaranteed to end it by Observation A.17. $\square$

We are now ready to prove that our adopt-commit protocol satisfies termination, validity and agreement.

Theorem A.20 (Termination). *Every correct process that calls Propose in a certain execution of KCensus (Algorithm 2) adopts or commits.*

Proof. We show that every correct process that calls Propose returns from this call. The theorem then follows, since a Propose invocation adopts or commits immediately before returning (on either line 13 or line 19).

If some correct process receives a Freeze message, it sends a DoAdopt message to all on line 37. By our network model, which assumes perfect links, all correct processes eventually receive a DoAdopt message. By Claim A.19, we are done.

Otherwise, no correct process receives a Freeze message, thus they all handle any received $\text{Accept\&Spread}$ message. If no correct process calls Propose, termination is vacuously satisfied, hence, assume at least one correct process calls Propose. Then by Lemma A.3, every correct process eventually receives an $\text{Accept\&Spread}$ message. We consider two distinct cases.

In the first case, two correct processes receive different values in their first received $\text{Accept\&Spread}$ message. By Observation A.2, each of them sends its received value to others via an $\text{Accept\&Spread}$ message (on line 33). By our network model, which assumes perfect links, all correct processes eventually receive these conflicting $\text{Accept\&Spread}$ messages. Each correct process, upon receiving the first $\text{Accept\&Spread}$ message, sets its accepted to the included value on line 24. When it receives an $\text{Accept\&Spread}$ message with a different value, it sends all—including itself—a DoAdopt message on line 26. By Claim A.19, its Propose call returns.

In the complementary case, all correct processes receive the same value in their first received Accept&Spread message. Let this value be v . Consider some correct process r that calls Propose. If it proposes a value different from v , then it sends an Accept&Spread message with its value to itself on line 10, and upon handling it on line 23, it sends all—including itself—a DoAdopt message on line 26. By Claim A.19, its Propose call returns. Otherwise (r proposes value v), since by Claim A.16 it eventually reaches a configuration from which calling `can_commit( $v$ )` would return True, then we are done by Observation A.18. $\square$

Theorem A.21 (Validity). *Every Propose call that returns in a certain execution of KCensus, adopts or commits a value that was the input of some Propose call.*

Proof. Propose may adopt or commit at one of two points. It can commit its input value on line 13, in which case validity is trivially satisfied. Alternatively, it can adopt on line 19 either a value returned by the `potential_commit` call on line 17 or some proposed value picked on line 18. In the latter case we are done. In the first case, `potential_commit` returns a non- $\perp$ value appearing in one of the Frozen replies received at the process (lines 52–53). A Frozen message contains the value that appears in the accepted local variable at the sending process (line 38). The value of `accepted` is set only on line 24, to a value received in an Accept&Spread message. By Observation A.1, such a value is a proposed value. $\square$

Theorem A.22 (Agreement). *If some Propose call in a certain execution of KCensus commits v , then all Propose calls that return adopt or commit value v .*

Proof. Suppose a proposer r commits its value v . Let r' be any other process that returns from a Propose call, and let v' be the value it commits or adopts. We need to show that $v' = v$. Since r commits v (which may occur only on line 13), it accepted v according to line 42. By Corollary A.9, all processes in `REQUIREMENTS[ $r$ ].keys()` have assigned value v to their accepted local variable.

If r' commits, then following the same reasoning as for r , r' accepted v' and all processes in `REQUIREMENTS[ $r'$ ].keys()` have assigned value v' to their accepted local variable. But as we assume a compatible combination of requirements, `REQUIREMENTS[ $r$ ].keys()` and `REQUIREMENTS[ $r'$ ].keys()` intersect by Observation A.10. Consider a process in the intersection. We showed that it should have assigned both values v and v' to its accepted local variable. Since each process accepts at most one value by Observation A.4, $v' = v$ in this case.

Otherwise, r' adopts v' . We next show that r' obtains the adopted v' on line 17 through a call to `potential_commit` that returns v , hence $v' = v$. First observe that by Lemma A.12, calling `could_potentially_commit` on line 53 with any $v^* \neq v$ returns False. Additionally, r' receives a Frozen reply with value v by Lemma A.13. Consequently, after calls to `could_potentially_commit` return False in previous iterations of the for loop on line 52, r' reaches an iteration in which it calls `could_potentially_commit` with v . By Lemma A.14, this call returns True. Thus, the `potential_commit` call at r' returns v . $\square$

A.2 KCensus's Correctness

KCensus uses the consensus protocol in Algorithm 7 with the implementation of adopt-commit appearing in Section 4. We prove that Algorithm 7 satisfies the consensus properties presented in the beginning of this section.

Theorem A.23. *Algorithm 7 satisfies termination, validity and agreement.*

Proof. The correctness of the algorithm is derived from the correctness of adopt-commit and the fallback consensus.

For termination, the call to `AdoptCommit::Propose` (line 2) in a correct process must result in a Commit or Adopt event, by adopt-commit's termination property. In case of commit, the consensus protocol decides on line 4. In case of adopt, the fallback consensus is invoked (line 6). Since the fallback protocol is guaranteed to decide (by its termination property), the main consensus protocol must also decide (line 8).

As for validity, adopt-commit is invoked only with values proposed in the consensus protocol (line 2). By adopt-commit's validity property, a call to `AdoptCommit::Propose` can only result (unless crashed) in adopting or committing a value that was previously proposed within adopt-commit, hence proposed by the consensus protocol. In case of commit, the consensus protocol decides that value (line 4), and we are done. In case of adopt, the fallback consensus is invoked with that value (line 6). In the broader view, all processes whose adopt-commit calls trigger adoption will proceed (unless crashed) to invoke the fallback consensus with the value they adopted, which is a value originally proposed in the consensus protocol. By its validity property, the fallback protocol is guaranteed to decide a value proposed for it (which is, as explained, a value proposed in the consensus protocol). Thus, the main consensus protocol must also decide a value proposed for it (line 8) (unless crashed).

Finally, we prove agreement. If the adopt-commit invocation does not result in commit in any process, then no process executes line 4. All processes that complete the adopt-commit invocation, invoke the fallback consensus protocol (line 6), and decide the same value for the main consensus protocol (line 8) by the fallback consensus's agreement (unless crashed). Otherwise, assume the adopt-commit invocation on some process commits some value v . By adopt-commit's agreement, the

adopt-commit invocation on all processes may adopt or commit only v . Every process may decide on line 4 or line 8. In the first case, it decides the committed value v . In the second case, since all processes that adopt invoke the fallback protocol with value v , then by the fallback consensus's validity, they decide v for the main consensus protocol (line 8). $\square$

B KCensus Optimality

We start by defining some terms used in our optimality proof.

Definition 1 (happened-before [53]). *Happened-before denotes the smallest relation on the set of steps of an execution satisfying the following three conditions:*

1. *If a and b are steps of the same process and a comes before b in the execution, then a happened-before b .*
2. *If a is the sending of a message by one process and b is the receipt of the same message by another process, then a happened-before b .*
3. *If a happened-before b and b happened-before c , then a happened-before c .*

Definition 2 (indistinguishable). *Two executions e_1, e_2 are indistinguishable to some process r until time t (including), if in both executions,*

1. *Propose is invoked at r at the same time $\leq t$ or not invoked at r at a time $\leq t$.*
2. *r receives the same messages at the same times until time t (including).*

We denote this relationship as $e_1 \overset{r}{\approx}_t e_2$. We similarly denote executions indistinguishable to some process r until time t excluding as $e_1 \overset{r}{\approx}_{<t} e_2$.

If the executions are indistinguishable to r at all times, we omit the time indication: $e_1 \overset{r}{\approx} e_2$.

Definition 3 (known message). *We define a message m in an execution to be known by process r at time t , if either m is received by r up to time t , or m 's receipt happened-before some message receipt at r occurring up to time t . We say in this case that r knows about m at t .*

We assume an eventually-stable network. In this section, we prove optimality in this eventual state of stability (defined below), but the proof could be easily extended to earlier long-enough periods of synchrony.

Definition 4 (stable network). *A network is stable if all inter-process latencies are stable and known to processes, all faulty processes are permanently down, and weak failure detectors in the system suspect all faulty processes while they do not suspect any correct process as having crashed.*

Definition 5 (weighted objective latency). *We define the weighted objective latency of a consensus protocol, in a stable network setting, to be the objective-function time from proposal to decision, where the objective function is taken over the set of executions in which Propose is invoked on a single correct process, weighted by the probability that each correct process is the proposer. The objective function could be the expectation (yielding the weighted expected latency), or a percentile (yielding weighted tail or median latency).*

Definition 6 (optimal combination of requirements of KCensus). *Given an objective latency function, an optimal combination of requirements is any assignment to REQUIREMENTS of KCensus, which results in the minimal weighted objective latency achievable by KCensus for a given stable network across all combinations of requirements that satisfy KCensus's conditions on requirements. Details on how to obtain it appear in Section 5.2.*

When using KCensus, assuming a certain objective latency function, KCensus is run with an optimal combination of requirements. In the following proof, we sometimes run the KCensus protocol with other combinations of requirements just for the sake of the proof.

The executions built in the proofs below suppress some messages, e.g., all those a process sends from some point on. A suppressed message is never dropped in violation of the model: it is either sent by a process that crashes later in that execution, or delayed until after every correct process has decided, which unbounded delays allow. Each suppression removes a suffix of the messages a link carries, so in-order delivery is preserved.

We are ready to present our main optimality result:

Theorem B.1 (Optimality). *Given a certain objective latency function, KCensus has optimal weighted objective latency in a stable network setting across all deterministic consensus protocols.*

Assumption: We consider consensus protocols in which some Propose invocation happened-before any message sending. This assumption is without loss of generality for the executions we compare: a message sent with no proposal happening before it does not depend on the system input, so it is sent identically and can be delivered at the same times in all of them, and it carries no information about a proposed value. We rely on this assumption when deriving AlgReqs below, where every message known by a proposer when it decides is read as spreading its value.

Proof. Let Alg be any deterministic consensus protocol. Assume a given weighted objective latency. We show that when the network is stable, Alg cannot have better weighted objective latency than KCensus.

Consider a stable network with up to f faulty processes. Let $\mathcal{F}$ indicate the set of faulty processes (the rest of the processes are correct). Let $\mathcal{L}$ indicate inter-process latencies in the stable network for all pairs of correct processes, where $\mathcal{L}[s, p]$ indicates the inter-process latency from s to p . We assume, as is standard, that message transmission takes a strictly positive amount of time, except when a process sends a message to itself, which is assumed to take zero time. Namely, $\mathcal{L}[s, p] > 0$ for $s \neq p$ and $\mathcal{L}[p, p] = 0$.

For simplicity, we intuitively assume that independent events cannot happen at a process at the same time. Formally, we assume that a process p does not send messages right before (at the exact same time) it receives a message from another process, and we assume that p cannot receive more than one message from other processes at the same time. Namely, at any given time, p may receive at most one message from another process, then it can send messages, then possibly receive a message from itself, followed by another round of message sending; the last two steps may repeat multiple times. This assumption implies that if a process p receives a message from another process at some time t , then this receipt is the first message receipt or sending event by p at t . This will be important in the definition and usage of $t_{p,r}$ below.

We define a combination of requirements AlgReqs by considering, for each correct process r , an execution where Propose is invoked on r only, and using it to define the requirement AlgReqs[r]. This is done in the following manner. For each process $r \notin \mathcal{F}$, let e_r be a single-proposer execution of Alg in which Alg.Propose(v_r) is invoked on r at time 0—where v_r is a unique value associated with each r , and the network is stable such that processes in $\mathcal{F}$ are crashed, and message transmission among correct processes is according to the latencies of $\mathcal{L}$. By termination, r must decide. Consider all messages known by r when r decides. Process these messages using Algorithm 8 in order of their reception time to compute known_acceptors for all processes that receive any of these messages. Set AlgReqs[r] to array_to_dict(known_acceptors[r]), where the value of known_acceptors[r] is obtained after processing all these messages in Algorithm 8, and array_to_dict (line 11) transforms it into a dictionary (each process p for which known_acceptors[r][p] is not empty becomes a key, with known_acceptors[r][p] as its associated value).

Algorithm 8. Knowledge Tracking: algorithm for execution e_r of proposer r

```

1 known_acceptors: array[n][n] of set<pid> = {0} // Initialize all array cells to empty sets
2 known_acceptors[r][r] = {r}
3 M = all messages in  $e_r$  known by  $r$  when  $r$  decides

5 for each message  $m$  from  $s$  to  $t$  in M, sorted by reception time:
6   Let knowledge_of_s be the value of known_acceptors[s] right after processing the last message in M received by  $s$ 
    $\hookrightarrow$  prior to sending  $m$  (or its initial value if no such message exists).
7   known_acceptors[t][t]  $\cup$ = { $t$ }  $\cup$  knowledge_of_s[s]
8   for  $q$  in pids:
9     known_acceptors[t][q]  $\cup$ = knowledge_of_s[q]

11 def array_to_dict(known_acceptors_of_process):
12   dict = {}
13   for  $p$  in pids:
14     if known_acceptors_of_process[p]  $\neq$  0:
15       dict[p] = known_acceptors_of_process[p]
16   return dict

```

We prove in Claims B.2–B.4 that every derived requirement AlgReqs[r] has the normal form of §3.1, is valid, and is pairwise compatible with the other derived requirements. Hence, KCensus could use AlgReqs. Let $KCensus_{AlgReqs}$ be KCensus with the combination of requirements AlgReqs. We show in Claim B.5 below that when the network is stable (with faulty processes $\mathcal{F}$ and inter-process latencies $\mathcal{L}$), Alg does not have better weighted objective latency than $KCensus_{AlgReqs}$. Since KCensus uses

an optimal combination of requirements, $KCensus_{AlgReqs}$ cannot have better weighted objective latency than $KCensus$ for the same stable network. Overall, we get that when the network is stable, Alg does not have better weighted objective latency than $KCensus$. This implies that $KCensus$ is a consensus protocol with optimal weighted objective latency. $\square$

We define some notations that will be used in the following proofs, in addition to the notations from the proof of Theorem B.1. Henceforth, references to a process r for which an execution e_r exists, or for which $AlgReqs[r]$ is defined, implicitly assume that r is correct. Consider some process r . Let t_r be the time at which r decides in e_r . Let $K_r = AlgReqs[r].keys()$. For each process $p \in K_r$, let $t_{p,r}$ be the time in which p receives in e_r its first message from another process that is not known by r at time t_r , or $t_r + \epsilon$ if no such message exists, where ϵ is some positive number. (The reason we pick the first message from another process is that self messages keep executions, which were indistinguishable to a process until receiving them, indistinguishable; so there is no need to discard them to obtain indistinguishable executions. We later define executions in which we discard all messages from other processes not known by r at t_r . To this end, we discard all the messages to / from process p that p does not receive / send before time $t_{p,r}$. All messages p receives / sends at $t_{p,r}$ or later are indeed not known by r at t_r , thanks to the fact that the receipt of the message from another process by p at $t_{p,r}$ is the first message receipt or sending event by p at $t_{p,r}$, combined with Observation B.7.)

Next, we bring the main claims used in our optimality proof, followed by lemmas used to prove them.

Claim B.2. *For each process r , the dictionary $AlgReqs[r]$ satisfies: (1) every key appears within its value; (2) all processes in the values appear as keys; and (3) r is a key and $AlgReqs[r][r] = AlgReqs[r].keys()$.*

Proof. Let r be some process. We prove by induction on messages in e_r known by r at t_r , by the order they are processed in Algorithm 8, that for every process p , conditions (1) and (2) hold for the dictionary $array_to_dict(known_acceptors[p])$ right after the message is processed. As $AlgReqs[r]$ equals the value of $array_to_dict(known_acceptors[r])$ after processing all these messages, QED follows.

For the base case, before any message is handled, all sets in $known_acceptors[p]$ for any process $p \neq r$ are empty, hence the transformation of $known_acceptors[p]$ into a dictionary yields an empty dictionary, that trivially satisfies conditions (1) and (2). As for process r , the only non-empty set in $known_acceptors[r]$ at initialization is $known_acceptors[r][r]$ which equals $\{r\}$. Hence, the transformation of $known_acceptors[r]$ into a dictionary yields a dictionary with r as the only key, having $\{r\}$ as its associated value. This dictionary clearly satisfies conditions (1) and (2).

Suppose the statement holds until right before some process t receives from some process s a message m known by r at t_r . When processing m in Algorithm 8, $known_acceptors[t]$ is updated. By the inductive hypothesis, $array_to_dict(known_acceptors[t])$ for the value of $known_acceptors[t]$ right before processing m , and $array_to_dict(known_acceptors[s])$ (where $known_acceptors[s]$ is the value of $known_acceptors[s]$ right after processing the last message, known by r at t_r , received by s prior to sending m , or the initial value of $known_acceptors[s]$ if no such message exists), satisfy conditions (1) and (2).

We first show that condition (1) is satisfied after processing m . For any key that already existed in $array_to_dict(known_acceptors[t])$ before processing m , condition (1) holds by the inductive hypothesis (as mentioned above). If t is added to $array_to_dict(known_acceptors[t])$ on line 7 (which happens if $known_acceptors[t][t]$ is empty before processing m), condition (1) holds since t is added to $known_acceptors[t][t]$. For any key q added to $array_to_dict(known_acceptors[t])$ on line 9 (q is added if $known_acceptors[t][q] = \emptyset$ before processing m while $known_acceptors[s][q] \neq \emptyset$), $q \in known_acceptors[s][q]$ due to the inductive hypothesis (as mentioned above), and is thus added to $known_acceptors[t][q]$.

We proceed to show that condition (2) is satisfied after processing m . The processes that may be added to sets in $known_acceptors[t]$ when processing m , are t (on line 7) and processes appearing in the sets in $known_acceptors[s]$ (on lines 7 and 9). t is guaranteed to be a key in $array_to_dict(known_acceptors[t])$ after processing m due to line 7. As for processes appearing in the sets in $known_acceptors[s]$, due to the inductive hypothesis (as mentioned above), each process p appearing in some set in $known_acceptors[s]$ is a key in $array_to_dict(known_acceptors[s])$, i.e., $known_acceptors[s][p] \neq \emptyset$. Hence, p is guaranteed to be a key in $array_to_dict(known_acceptors[t])$ after line 9 is executed with $q = p$.

It remains to prove (3). Process r is a key because $known_acceptors[r][r]$ is initialized to $\{r\}$. Condition (2) gives $AlgReqs[r][r] \subseteq AlgReqs[r].keys()$. Since $known_acceptors[r]$ changes only when r receives a message, the reverse inclusion follows from Lemma B.16 applied to the last processed message received by r , or from initialization if there is none. $\square$

Claim B.3. *The requirement $AlgReqs[r]$ of each process r is valid (see definition in Algorithm 1).*

Proof. Assume, for sake of contradiction, that there exists some process r whose associated requirement, $AlgReqs[r]$, is not valid. This means that $|K_r| \leq f$.

Let $\mathcal{L}'$ indicate inter-process latencies for all pairs of processes not in K_r in the following way: $\mathcal{L}'[p, p] = 0$ for any $p \notin K_r$, and for every two processes $s, p \notin K_r$ such that $s \neq p$, $\mathcal{L}'[s, p] = \mathcal{L}[s, p]$ if $s, p \notin \mathcal{F}$, else $\mathcal{L}'[s, p]$ is assigned an arbitrary positive value. Let r' be some process $\notin K_r$.

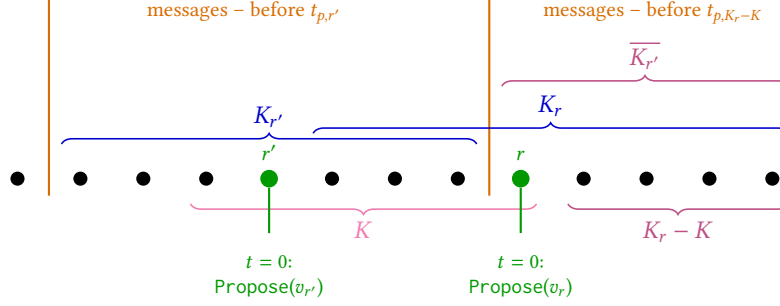


Figure 15. Illustration of execution f_1 until time t_d : Dots represent processes. Proposals are shown in green. The different sets that processes belong to are illustrated in blue or pink. The communication between any process p and the others until time t_d in f_1 is illustrated in orange.

Consider a single-proposer execution e_1 of Alg, in which:

- Alg.Propose($v_{r'}$) is invoked on process r' at time 0.
- Only the processes in K_r are faulty, and they all crash at time 0.
- Message transmission among processes not in K_r is according to the latencies of $\mathcal{L}'$.

Further consider an execution e_2 of Alg, in which:

- Alg.Propose(v_r) is invoked on process r at time 0.
- Alg.Propose($v_{r'}$) is invoked on process r' at time 0.
- Only the processes in K_r are faulty, and they all crash right after time t_r .
- Message transmission among processes in K_r is according to the latencies of $\mathcal{L}$, except for any messages sent by any $p \in K_r$ at time $t_{p,r}$ or later, which are lost.
- Message transmission among processes not in K_r is according to the latencies of $\mathcal{L}'$.
- Messages sent from a process in K_r to a process not in that set, or vice versa, are lost.

We next show that in e_2 , r decides v_r while r' decides $v_{r'}$, contradicting the agreement property of the consensus protocol Alg. We start with r . By termination, r decides (at time t_r) in e_r . By validity, it decides v_r . By Corollary B.12, $e_2 \approx_{t_r}^r e_r$, hence r decides v_r (at time t_r) in e_2 as well. As for r' , by termination it decides in e_1 . By validity, it decides $v_{r'}$. By Lemma B.13, $e_2 \approx_{t_r}^{r'} e_1$, hence r' decides $v_{r'}$ in e_2 as well. $\square$

Claim B.4. The combination of requirements AlgReqs is compatible (see definition in Algorithm 1).

Proof. Assume, for sake of contradiction, that there exist processes r, r' such that for at most f processes in K_r or $K_{r'}$ (or both), the corresponding value (namely $\text{AlgReqs}[r][p]$ or $\text{AlgReqs}[r'][p]$ respectively for process p) contains some process in the intersection $K_r \cap K_{r'}$. Denote this set of at most f processes (about which r knows at t_r in e_r that they know about some process in the intersection, or r' knows at $t_{r'}$ in $e_{r'}$ that they know about some process in the intersection) by K . Let $\overline{K_{r'}} = K_r - K_{r'}$. Let t_{p,K_r-K} be the time in which process $p \in \overline{K_{r'}}$ receives in e_r its first message from another process that is not known by any process $q \in K_r - K$ before time $t_{q,r}$, or $t_{p,r}$ if no such message is received until $t_{p,r}$. (Note that for $p \in K_r - K$, $t_{p,K_r-K} = t_{p,r}$.) Let $t_d = \max\{t_r, t_{r'}\} + \epsilon$. Further let $\mathcal{L}^*$ indicate inter-process latencies for all pairs of processes not in K in the following way: $\mathcal{L}^*[p, p] = 0$ for any $p \notin K$, and for every two processes $s, p \notin K$ such that $s \neq p$, $\mathcal{L}^*[s, p] = \mathcal{L}[s, p]$ if $s, p \notin \mathcal{F}$, else $\mathcal{L}^*[s, p]$ is assigned an arbitrary positive value.

We next design f_1 (illustrated in Figure 15) and f_2 , which will satisfy the following: (1) f_1 and $e_{r'}$ are indistinguishable to r' until it decides, so that r' decides $v_{r'}$ in f_1 . (2) f_2 and e_r are indistinguishable to r until it decides, so that r decides v_r in f_2 . (3) f_1 and f_2 are indistinguishable to all processes not in K , so that they decide (by termination) the same value in both executions. At the same time, by agreement, they should decide $v_{r'}$ in f_1 , but v_r in f_2 , which yields the desired contradiction.

Intuitively, to achieve (1), we design f_1 as follows: we invoke a proposal of $v_{r'}$ on r' at time 0, and let any process $p \in K_{r'}$ receive from other processes messages known by r' at $t_{r'}$ in $e_{r'}$ and only them until time $t_{r'}$. Specifically, p only receives messages prior to $t_{p,r'}$ and messages p sends starting at $t_{p,r'}$ are lost; and any messages among processes in $K_{r'}$ and other processes are discarded. To achieve (2), we design f_2 symmetrically for r .

To achieve (3) for any process $p \in K_r - K$, since in f_2 , until time t_r , p receives from other processes messages known by r at t_r in e_r and only them, in f_1 we also have p receive from other processes messages known by r at t_r in e_r and only them until time t_r . As $p \notin K$, r does not have knowledge at t_r in e_r that p knows about any process in the intersection $K_r \cap K_{r'}$.

This means that p should not know in f_1 about processes in the intersection. We achieve that intuitively by having p , until time t_r in f_1 , receive from processes in $\overline{K_{r'}} \cap K$ only messages sent before they learned about the intersection, and not receive any messages from processes in $K_r \cap K_{r'}$. More concretely, until time t_r in f_1 , we let any process $p \in \overline{K_{r'}}$ receive from other processes messages known by r at t_r in e_r and only them, by enforcing the following: every process $p \in K_r - K$ only receives messages prior to $t_{p,r}$ and messages p sends starting at $t_{p,r}$ are lost; every process $p \in \overline{K_{r'}} \cap K$ only receives messages prior to t_{p,K_r-K} and messages p sends starting at t_{p,K_r-K} are lost; and any messages among processes in $\overline{K_{r'}}$ and other processes are discarded. To achieve (3) for processes in $K_{r'} - K$, we design f_2 symmetrically for r' .

We now formally define f_1 and f_2 .

Consider an execution f_1 of Alg, in which:

- Alg.Propose(v_r) is invoked on r at time 0.
- Alg.Propose($v_{r'}$) is invoked on r' at time 0.
- Only the processes in K are faulty, and they all crash right after time t_d .
- Message transmission of messages sent until time t_d (including) is according to the latencies of $\mathcal{L}$, except for the following: (Intuitively, the first and third following items ensure that $f_1 \stackrel{p}{\approx}_{<t_{p,r'}} e_{r'}$ for every $p \in K_{r'}$ and, in particular, $f_1 \stackrel{r'}{\approx}_{t_r} e_{r'}$; and the second and third items ensure that $f_1 \stackrel{p}{\approx}_{<t_{p,K_r-K}} e_r$ for every $p \in \overline{K_{r'}}$ and, in particular, $f_1 \stackrel{p}{\approx}_{<t_{p,r}} e_r$ for every $p \in K_r - K$.)
 - For each process $p \in K_{r'}$, any messages sent to p before t_d , which do not reach it before $t_{p,r'}$ according to the latencies of $\mathcal{L}$, are lost. (This implies that no messages are received by p in f_1 since time $t_{p,r'}$ until t_d (including).) In addition, all messages sent by p at time $t_{p,r'}$ or later until t_d (including), are lost.
 - For each process $p \in \overline{K_{r'}}$, any messages sent to p before t_d , which do not reach it before t_{p,K_r-K} according to the latencies of $\mathcal{L}$, are lost. (This implies that no messages are received by p in f_1 since time t_{p,K_r-K} until t_d (including).) In addition, all messages sent by p at time t_{p,K_r-K} or later until t_d (including), are lost.
 - Messages between two processes that are not both in $K_{r'}$ or both in $\overline{K_{r'}}$ are lost.
 - Messages to processes in K that should reach them after t_d according to the latencies of $\mathcal{L}$, do not reach their target because it is down.
- Message transmission of messages sent after time t_d is according to the latencies of $\mathcal{L}^*$, except for messages to processes in K which do not reach their target because it is down.

Let $\overline{K_r} = K_{r'} - K_r$. Let $t_{p,K_{r'}-K}$ be the time in which process $p \in \overline{K_r}$ receives in $e_{r'}$ its first message from another process that is not known by any process $q \in K_{r'} - K$ before time $t_{q,r'}$, or $t_{p,r'}$ if no such message is received until $t_{p,r'}$. (Note that for $p \in K_{r'} - K$, $t_{p,K_{r'}-K} = t_{p,r'}$.) Consider an execution f_2 of Alg, which is symmetrical to f_1 , obtained by exchanging the roles of r and r' . In detail, in f_2 :

- Alg.Propose(v_r) is invoked on r at time 0.
- Alg.Propose($v_{r'}$) is invoked on r' at time 0.
- Only the processes in K are faulty, and they all crash right after time t_d .
- Message transmission of messages sent until time t_d (including) is according to the latencies of $\mathcal{L}$, except for the following: (Intuitively, the first and third following items ensure that $f_2 \stackrel{p}{\approx}_{<t_{p,r}} e_r$ for every $p \in K_r$ and, in particular, $f_2 \stackrel{r}{\approx}_{t_r} e_r$; and the second and third items ensure that $f_2 \stackrel{p}{\approx}_{<t_{p,K_{r'}-K}} e_{r'}$ for every $p \in K_{r'} - K$.)
 - For each process $p \in K_r$, any messages sent to p before t_d , which do not reach it before $t_{p,r}$ according to the latencies of $\mathcal{L}$, are lost. (This implies that no messages are received by p in f_2 at time $t_{p,r}$ or later until t_d (including).) In addition, all messages sent by p at time $t_{p,r}$ or later until t_d (including), are lost.
 - For each process $p \in \overline{K_r}$, any messages sent to p before t_d , which do not reach it before $t_{p,K_{r'}-K}$ according to the latencies of $\mathcal{L}$, are lost. (This implies that no messages are received by p in f_2 at time $t_{p,K_{r'}-K}$ or after until t_d (including).) In addition, all messages sent by p at time $t_{p,K_{r'}-K}$ or after until t_d (including), are lost.
 - Messages sent until time t_d (including), between two processes that are not both in K_r or both in $\overline{K_r}$, are lost.
 - Messages to processes in K that should reach them after t_d according to the latencies of $\mathcal{L}$, do not reach their target because it is down.
- Message transmission of messages sent after time t_d is according to the latencies of $\mathcal{L}^*$, except for messages to processes in K which do not reach their target because it is down.

By Lemma B.19, $r \notin K_{r'}$ and $r' \notin K_r$. Additionally, by Lemma B.20, $r, r' \in K$. Consequently, $r \in K - K_{r'}$ and $r' \in K - K_r$.

We next show that in f_2 , r decides v_r while processes not in K decide $v_{r'}$, contradicting the agreement property of the consensus protocol Alg. We start with r . By termination, r decides (at time t_r) in e_r . By validity, it decides v_r . It could be proven, just like proven in Corollary B.12 for e_2 , that $f_2 \stackrel{r}{\approx}_{t_r} e_r$, hence r decides v_r (at time t_r) in f_2 as well. As for r' , by termination, r' decides (at time $t_{r'}$) in $e_{r'}$. By validity, it decides $v_{r'}$. It could be proven, just like proven in Corollary B.12 for e_2 , that $f_1 \stackrel{r'}{\approx}_{t_{r'}} e_{r'}$, hence r' decides $v_{r'}$ (at time $t_{r'}$) in f_1 as well. By agreement and termination, all processes not in K decide $v_{r'}$ in f_1 . By Lemma B.23, $f_2 \stackrel{p}{\approx} f_1$ for any process $p \notin K$, hence these processes decide $v_{r'}$ in f_2 as well. $\square$

Claim B.5. *The weighted objective latency of Alg is not smaller than that of $KCensus_{AlgReqs}$.*

Proof. For every process $r \notin \mathcal{F}$, let e_r^* be a single-proposer execution of $KCensus_{AlgReqs}$ in which $KCensus_{AlgReqs}.Propose(v_r)$ is invoked on r at time 0, and the network is stable such that processes in $\mathcal{F}$ are crashed, and message transmission among correct processes is according to the latencies of $\mathcal{L}$. We next show that r decides at time $\leq t_r$ in e_r^* . As r decides at time t_r in e_r , which is the equivalent execution of Alg, then the latency of Alg is not smaller than that of $KCensus_{AlgReqs}$ for r as proposer. Taking the weighted objective latency over executions for each correct process as the proposer, we get QED.

By termination, r decides in e_r^* . Hence, the $Propose(v_r)$ call invoked on r must break from the wait on line 11. Since no $DoAdopt$ messages are received in e_r^* by Lemma B.25, the $Propose(v_r)$ call breaks from the wait on line 11 only when $can_commit(v_r)$ returns True. If $can_commit(v_r)$ by r does not return True in e_r^* before time t_r , then at t_r it does by Lemma B.28. This triggers a commit event in the adopt-commit protocol (line 13), which in turn triggers a decision event in the consensus protocol on r (line 4). $\square$

Before presenting the main lemmas used in the proofs of the above claims, we bring several supporting observations and lemmas that will facilitate the proof of these main lemmas. These statements characterize the processes in K_r as well as values in $known_acceptors$.

Observation B.6. *For any process r , $r \in K_r$.*

Proof. $known_acceptors[r][r]$ is initialized to a non-empty set on line 2. This adds the key r to $array_to_dict(known_acceptors[r])$. The dictionary $array_to_dict(known_acceptors[r])$ is only ever extended. As $AlgReqs[r]$ is assigned the value of $array_to_dict(known_acceptors[r])$ after completing the run of Algorithm 8, $r \in AlgReqs[r].keys() = K_r$. $\square$

Observation B.7. *Consider a process r . A message m , received by some process p in e_r before p receives or sends some message m' —which is known by r at t_r , is also known by r at t_r .*

Proof. Message m' must be received by its target, as it is known by r . Clearly, m 's receipt happened-before m' 's receipt. Additionally, as m' is known by r at t_r , m' is received by r up to time t_r , or the receipt of m' happened-before the receipt of some message m^* at r occurring up to time t_r . In the first case, m 's receipt happened-before m' 's receipt by r occurring at time $\leq t_r$, and in the latter case— m 's receipt happened-before m^* 's receipt by r occurring at time $\leq t_r$. Thus, m is known by r at time t_r . $\square$

Thanks to our assumptions that at a certain time a process p may receive a single message from another process (possibly followed by messages from p to itself); and that p does not send messages right before receiving a message from another process at the exact same time—we may deduce the following:

Corollary B.8. *Consider a process r . Messages sent in e_r by any process $p \in K_r$ at time $t_{p,r}$ or later are not known by r at t_r .*

Lemma B.9. *Consider some process r . For any message known by r at t_r , received by some process p in e_r , the following holds after processing the message in Algorithm 8:*

- (1) $p \in array_to_dict(known_acceptors[p]).keys()$,
- (2) all of the keys in $array_to_dict(known_acceptors[p])$ are in K_r , and
- (3) $known_acceptors[p][q] \subseteq AlgReqs[r][q]$ for any process q .

Proof. We prove by induction on messages in e_r known by r at t_r , ordered by the reverse of their processing order in Algorithm 8. For the base case, consider the last message m , known by r at t_r , received in e_r up to time t_r . m must be received by r , because if it is received by another process, its receipt must be followed by sending another message that is known by r at t_r , which is received up to time t_r —contradicting the receipt of m being last. Condition (1) holds due to line 2. Conditions (2) and (3) hold because $AlgReqs[r]$ is set to the value of $array_to_dict(known_acceptors[r])$ after processing m .

Now suppose the statement holds for messages, known by r at t_r , received after some message m , known by r at t_r , is received by process p . As $m \in M$ for M defined on line 3, m is processed in the for loop on line 5. On line 7, the key p is added to the $array_to_dict(known_acceptors[p])$ dictionary (if it is not yet there), hence condition (1) holds.

As for conditions (2) and (3): if $p = r$, then since all processes in $\text{array_to_dict}(\text{known_acceptors}[r]).\text{keys}()$ after processing m remain in it until t_r , they are in K_r and condition (2) holds. Similarly, for any process q , all processes in $\text{known_acceptors}[r][q]$ after processing m remain in it until t_r , hence, they are in $\text{AlgReqs}[r][q]$ and condition (3) holds. Else ($p \neq r$), as m is a message known by r at t_r , then after receiving m , p must send to some process p^* some message m^* , known by r at t_r . We start with condition (2): All processes that were in $\text{array_to_dict}(\text{known_acceptors}[p]).\text{keys}()$ after m is processed in Algorithm 8, remain there until right after processing the last message in M received by p prior to sending m^* . After processing m^* , all processes that were in $\text{array_to_dict}(\text{known_acceptors}[p]).\text{keys}()$ after processing m , are in $\text{array_to_dict}(\text{known_acceptors}[p^*]).\text{keys}()$, due to lines 8–9. By the inductive hypothesis, all processes that were in $\text{array_to_dict}(\text{known_acceptors}[p^*]).\text{keys}()$ after processing m^* are in K_r . Similarly, for condition (3): All processes that were in $\text{known_acceptors}[p][q]$ for any process q after processing m remain there until right after processing the last message in M received by p prior to sending m^* . After processing m^* , all processes that were in $\text{known_acceptors}[p][q]$ for any process q after processing m , are in $\text{known_acceptors}[p^*][q]$, due to lines 8–9. By the inductive hypothesis, all processes that were in $\text{known_acceptors}[p^*][q]$ after processing m^* are in $\text{AlgReqs}[r][q]$. $\square$

Lemma B.10. *Consider some process r . Only processes in K_r receive and send in e_r messages known by r at t_r .*

Proof. We start with the case of a process p that receives a message known by r at t_r . From conditions (1) and (2) in Lemma B.9, it specifically follows that $p \in K_r$.

We proceed to the second case, of a process s sending in e_r a message m known by r at t_r . If $s = r$, then $s \in K_r$ by Observation B.6. Else, due to the assumption that a Propose invocation happened-before any message sending in Alg, and the fact that $r \neq s$ is the only proposer in e_r , s receives some message m^* before sending m . By Observation B.7, m^* is known by r at t_r . Then by the first case above ($p = s$ receives a message m^* known by r at t_r), $s \in K_r$. $\square$

The following lemmas are used in the proof of Claim B.3.

Lemma B.11. *Consider r chosen in the proof of Claim B.3 and e_2 defined in that proof. $e_2 \stackrel{p}{\approx}_{<t_{p,r}} e_r$ for any process $p \in K_r$.*

Proof. We prove that $e_2 \stackrel{p}{\approx}_t e_r$ for any time t such that a message is received by some process $p \in K_r$ at time $t < t_{p,r}$ in e_r or e_2 . This is enough, as e_2 and e_r remain indistinguishable to p from the last time p receives such a message up to time $t_{p,r}$ (since during this time frame, there are no additional message receipts by p in e_2 or e_r , nor any different proposals at p). We prove by induction on message receipt times. As Propose invocations on processes in K_r are the same in e_2 and e_r , it remains to show that each process $p \in K_r$ receives the same messages at the same times in both executions until any time $t < t_{p,r}$ (including t) in which it receives a message in one of the executions.

For the base case, we consider time 0. The base case trivially holds for all processes in K_r except for r : Due to the assumption that a Propose invocation happened-before any message sending in Alg, and the fact that proposals are invoked only at time 0 in e_2 and e_r , and only on r and $r' \notin K_r$, no messages are received by processes $\neq r$ in K_r up to time 0. Thus, e_2 and e_r are indistinguishable to these processes at time 0. As for r , it might receive messages from itself at time 0. In that case, e_2 and e_r are indistinguishable to r at time 0 before it receives the first message from itself. Hence, r sends the same message to itself in both executions, then receives it in both executions, and they remain indistinguishable to r after receiving the message. The same argument applies to any subsequent messages r sends itself at time 0. Therefore, e_2 and e_r are still indistinguishable to r at time 0 after receiving all of these self messages.

Suppose the statement holds prior to time $0 < t < t_{p,r}$ for some process $p \in K_r$, and p receives only messages from itself at time t in e_r and e_2 . Then by the same arguments as in the base case together with the inductive hypothesis, these messages are received in both executions and leave the executions indistinguishable to p . So suppose the statement holds prior to time $0 < t < t_{p,r}$ for some process $p \in K_r$, and p receives a message m at time t in e_r or e_2 from another process. Note that in case m is received in e_r , then since p receives its first message from another process that is not known by r at t_r (if any) at time $t_{p,r}$, m is known by r at t_r . For messages received by p in e_2 or e_r before time t , we may apply the inductive hypothesis, which yields that e_2 is indistinguishable to p from e_r until the receipt time of any such message. Hence, e_2 and e_r are indistinguishable to p prior to t . To show that e_2 and e_r are indistinguishable to p until t (including), we will show that m is received in both executions at time t . This is enough as by assumption, no message other than m is received by p at t from other processes; and messages p possibly receives from itself after m at time t are received in both executions and leave the executions indistinguishable to p , by the same arguments as in the base case.

Let s be the sender of m . To apply the inductive hypothesis to messages received by s in either execution until the time in which s sends m , we need to show they satisfy the conditions of the induction's statement: $s \in K_r$, and these messages are received before time $t_{s,r}$. For the latter, we show the stronger statement that s sends m before time $t_{s,r}$. We start with the case in which m is received in e_r . In this case, as m is known by r at t_r (as noted above), then $s \in K_r$ by Lemma B.10. Additionally, as

m is known by r at t_r , then by Corollary B.8, s sends m before time $t_{s,r}$. As for the case that m is received in e_2 , $p \in K_r$ receives only messages from processes in K_r in e_2 , hence $s \in K_r$. In addition, m is sent before $t_{s,r}$, as any messages s sends in e_2 at $t_{s,r}$ or later are lost.

So by the inductive hypothesis, e_2 and e_r are indistinguishable to s until receiving the last message (if any) in either execution up to the time in which it sends m . They hence remain indistinguishable to s from that moment (or from time 0 in case no message receipts by s precede the sending of m) until it sends m (in e_r or e_2). Therefore, m is sent in the other execution as well at the same time. Given that m originally appears in e_2 , it is not lost and reaches its target in e_r as well, since no messages to correct processes are lost in e_r . Given that m originally appears in e_r , it is not lost and reaches its target in e_2 as well, since m is a message from a process in K_r to a process in K_r , sent by s before $t_{s,r}$. Furthermore, m reaches its target at the same time in both executions—according to the corresponding latency in $\mathcal{L}$. That is, m is received in both executions at time t and we are done. $\square$

Corollary B.12. $e_2 \stackrel{r}{\approx}_{t_r} e_r$, for r chosen in the proof of Claim B.3 and e_2 defined in that proof.

Proof. $r \in K_r$ by Observation B.6. Additionally, $t_{r,r} = t_r + \epsilon$ by definition of $t_{r,r}$. Thus, applying Lemma B.11 to r yields QED. $\square$

Lemma B.13. $e_2 \stackrel{r'}{\approx}_{t_1} e_1$, for r' chosen in the proof of Claim B.3 and e_1 and e_2 defined in that proof.

Proof. We prove that $e_2 \stackrel{p}{\approx}_{t_1} e_1$ for each $p \notin K_r$. We start with Propose invocations. These are the same in e_2 and e_1 on all processes except for r , but $r \in K_r$ by Observation B.6.

We proceed to message receipts. In both executions—processes not in K_r do not receive messages from processes in K_r (up to time t_r they are lost in e_2 if sent, in other times the processes in K_r are down in both executions), and all messages among processes not in K_r are transmitted according to the latencies of $\mathcal{L}'$. Therefore, all processes not in K_r cannot distinguish between e_2 and e_1 , and communicate only among themselves (namely, with processes that do not distinguish as well). As Alg is deterministic, the two executions are indistinguishable to processes not in K_r at all times. $\square$

The following lemmas are used in the proof of Claim B.4.

Lemma B.14. Consider some processes r and x . Assume process p receives in e_r a message m , known by r at t_r . Further assume either m is sent by x or a sending of some message by x happened-before the sending of m . Then $x \in \text{known_acceptors}[p][p]$ right after handling m in Algorithm 8.

Proof. Consider some processes r and x . We prove by induction on the following messages, by the order they are processed in Algorithm 8: any message m in e_r , known by r at t_r , received by some process p , such that either m is sent by x or a sending of some message by x happened-before the sending of m .

For the base case, we consider any message m^* , known by r at t_r , sent by x to some process p . If $x \neq r$, then before sending m^* , x must have received a message, due to the assumption that a Propose invocation happened-before any message sending in Alg. This message is known by r at t_r by Observation B.7. Thus, it is processed in Algorithm 8. When it is processed, x is added to $\text{known_acceptors}[x][x]$ (if not already there) on line 7. If $x = r$, $\text{known_acceptors}[r][r]$ is initialized to $\{r\}$ on line 2. Whether $x = r$ or not, when m^* is processed in Algorithm 8, $x \in \text{knowledge_of_s}[x]$, and is thus added to $\text{known_acceptors}[p][p]$ (if not already there) on line 7.

Suppose the statement holds until some process p receives from process s a message m known by r at t_r , such that $s \neq x$ and a sending of some message by x happened-before the sending of m . Then before sending m , s must have received some message m^* , such that either m^* is sent by x or a sending of some message by x happened-before the sending of m^* . As m is known by r at t_r , then by Observation B.7, m^* is a message known by r at t_r . Consequently, we may apply the inductive hypothesis to m^* and conclude that $\text{known_acceptors}[s][s]$ contains x after handling m^* . Hence, when processing m in Algorithm 8, $\text{knowledge_of_s}[s]$ contains x . Thus, x is added to $\text{known_acceptors}[p][p]$ (if it was not already there) on line 7. $\square$

Lemma B.15. For any process r and any process p that receives in e_r a message m known by r at t_r , $r \in \text{known_acceptors}[p][p]$ right after handling m in Algorithm 8.

Proof. Due to the assumption that a Propose invocation happened-before any message sending in Alg, either m is sent by r or a message sending by r happened-before the sending of m . Hence, we may apply Lemma B.14 with $x = r$ to m . This yields QED. $\square$

Lemma B.16. For any process r and any process p that receives in e_r a message m known by r at t_r , $\text{array_to_dict}(\text{known_acceptors}[p]).\text{keys}() \subseteq \text{known_acceptors}[p][p]$ right after handling m in Algorithm 8.

Proof. Let r be some process. We prove by induction on messages in e_r known by r at t_r , by the order they are processed in Algorithm 8.

For the base case, before any message is handled, all sets in `known_acceptors` except for `known_acceptors[r][r]` are empty, hence the transformation of `known_acceptors[p]` for each process $p \neq r$ into a dictionary yields an empty dictionary. As for r , the only key in `array_to_dict(known_acceptors[r])` after initialization is r , and `known_acceptors[r][r] = {r}`.

Suppose the statement holds until some process t receives from process s a message m known by r at t_r . Let `knowledge_of_s` be the value of `known_acceptors[s]` right after processing the last message, known by r at t_r , received by s prior to sending m (or its initial value if no such message exists). By the inductive hypothesis, right before processing m in Algorithm 8, `array_to_dict(known_acceptors[t]).keys() ⊆ known_acceptors[t][t]`. The keys that are added to `array_to_dict(known_acceptors[t])` when processing m (if they are not already there) are t on line 7, and the keys in `array_to_dict(knowledge_of_s)` on line 9. t is added to `known_acceptors[t][t]` (if not already there) on line 7. `array_to_dict(knowledge_of_s).keys() ⊆ knowledge_of_s[s]` by the inductive hypothesis. Hence, the keys in `array_to_dict(knowledge_of_s)` are also added to `known_acceptors[t][t]` (if not already there) on line 7. $\square$

Lemma B.17. *For any process r , every process in K_r receives in e_r some message which is known by r at t_r .*

Proof. Let r be some process. We prove by induction on messages in e_r known by r at t_r , by the order they are processed in Algorithm 8, that for every process p that appears in `array_to_dict(known_acceptors[t]).keys()` after handling in Algorithm 8 a message m , known by r at t_r , received by some process t , the following holds: if $p \neq t$, then a message-sending event by p happened-before m 's receipt in e_r (namely, intuitively, there is a message chain from p to t ending with m). For any process $p \neq r$ in K_r , applying the induction's statement to m —the last message received by $t = r$ up to time t_r —yields QED: By the induction's statement, a sending of some message m' by p happened-before m 's receipt by r in e_r , thus, m' is known by r at t_r . Due to the assumption that a Propose invocation happened-before any message sending in Alg, and since a Propose is not invoked on $p \neq r$ in e_r , p received a message prior to sending m' . This received message is known by r at t_r by Observation B.7, and we are done. As for process r , it may decide in e_r only after receiving some message (otherwise the execution is indistinguishable to r from one in which another process communicates with all others before r does and decides another value).

We proceed with the proof by induction. For the base case, consider a message m from process s , which is known by r at t_r , received by process t . Let $p \neq t$ be a process that appears in `array_to_dict(known_acceptors[t]).keys()` after processing m in Algorithm 8, such that $p = s$ or $p = r$. We need to show that a message-sending event by p happened-before m 's receipt in e_r . If $p = s$ then we are trivially done. If $p = r$ then we are also done by assumption that a Propose invocation happened-before any message sending in Alg (including the sending of m).

Suppose the statement holds for all messages received before a message m from process s , which is known by r at t_r , is received by process t . Let $p \neq t, s, r$ be a process that appears in `array_to_dict(known_acceptors[t]).keys()` after processing m in Algorithm 8. We need to show that a message-sending event by p happened-before m 's receipt in e_r . If p was in `array_to_dict(known_acceptors[t]).keys()` before processing m , then we are done by the inductive hypothesis. Otherwise, p was added to `array_to_dict(known_acceptors[t]).keys()` when processing m on line 9 in Algorithm 8. This means that p was in `array_to_dict(known_acceptors[s]).keys()` right after processing the last message, known by r at t_r , received by s prior to sending m ; or p is in the initial value of `array_to_dict(known_acceptors[s]).keys()` if no such message exists. The latter is impossible: the initial value is an empty dictionary if $s \neq r$, else a dictionary that contains r as its only key—and $p \neq r$ cannot be a key in any of these dictionaries. Namely, before sending m , s must have received some message. Let m^* be the last message that s received before sending m . By Observation B.7, m^* is a message known by r at t_r . p , which was added to `array_to_dict(known_acceptors[t]).keys()` on line 9 when processing m , was in `array_to_dict(known_acceptors[s]).keys()` right after processing m^* . Applying the inductive hypothesis to m^* yields that since $p \neq s$, then a message-sending event by p happened-before m^* 's receipt by s . Consequently, a message-sending event by p happened-before m 's receipt by t . $\square$

Lemma B.18. *For any process r and process $p \in K_r$, $r \in \text{AlgReqs}[r][p]$.*

Proof. Consider some process r . Let p be a process in K_r . By Lemma B.17, p receives in e_r a message m known by r at t_r . Then by Lemma B.15, $r \in \text{known_acceptors}[p][p]$ right after handling m in Algorithm 8. By condition (3) in Lemma B.9, `known_acceptors[p][p] ⊆ AlgReqs[r][p]` after handling m . Therefore, $r \in \text{AlgReqs}[r][p]$. $\square$

Lemma B.19. *$r \notin K_{r'}$ and $r' \notin K_r$, for r and r' chosen in the proof of Claim B.4.*

Proof. Assume, for sake of contradiction, that $r \in K_{r'}$. As $r \in K_r$ by Observation B.6, we get $r \in K_r \cap K_{r'}$. For every process $p \in K_r$, $r \in \text{AlgReqs}[r][p]$ by Lemma B.18. Namely, `AlgReqs[r][p]` contains r —a process in the intersection $K_r \cap K_{r'}$. This implies that $K_r \subseteq K$ (for K defined in the proof of Claim B.4). $|K_r| > f$ by validity (Claim B.3), so $|K| > f$ —contradiction.

The proof for $r' \notin K_r$ is symmetrical. $\square$

Lemma B.20. $r \in K$, for r chosen in the proof of Claim B.4 and K defined in that proof.

Proof. Let p be a process in $K_r \cap K_{r'}$ for r and r' chosen in the proof of Claim B.4. By condition (3) of Claim B.2, $K_r = \text{AlgReqs}[r][r]$. We have $r \in K_r$ by Observation B.6, and $p \in K_r \cap K_{r'}$ satisfies $p \in \text{AlgReqs}[r][r]$. Hence, by definition of K , $r \in K$. $\square$

Lemma B.21. Consider r chosen in the proof of Claim B.4, and $f_1, K, \overline{K_{r'}}$ and t_{p,K_r-K} defined in that proof. $f_1 \stackrel{p}{\approx}_{<t_{p,K_r-K}} e_r$ for any process $p \in \overline{K_{r'}}$.

Proof. We prove that $f_1 \stackrel{p}{\approx}_t e_r$ for any time t such that a message is received in e_r or f_1 by some process $p \in \overline{K_{r'}}$ at time $t < t_{p,K_r-K}$. This is enough, as f_1 and e_r remain indistinguishable to p from the last time p receives such a message up to time t_{p,K_r-K} (since during this time frame, there are no additional message receipts by p in f_1 or e_r , nor any different proposals at p). We prove by induction on message receipt times. As Propose invocations on processes in $\overline{K_{r'}}$ are the same in f_1 and e_r , it remains to show that each p receives the same messages at the same times in both executions until time t (including).

For the base case, we consider time 0. The base case trivially holds for all processes in $\overline{K_{r'}}$ except for r : Due to the assumption that a Propose invocation happened-before any message sending in Alg, and the fact that proposals are invoked only at time 0 in f_1 and e_r , and only on r and $r' \notin \overline{K_{r'}}$, no messages are received by processes $\neq r$ in $\overline{K_{r'}}$ up to time 0. Thus, f_1 and e_r are indistinguishable to these processes at time 0. As for r , it might receive messages from itself at time 0. In that case, f_1 and e_r are indistinguishable to r at time 0 before it receives the first message from itself. Hence, r sends the same message to itself in both executions, then receives it in both executions, and they remain indistinguishable to r after receiving the message. The same argument applies to any subsequent messages r sends itself at time 0. Therefore, f_1 and e_r are still indistinguishable to r at time 0 after receiving all of these self messages.

Suppose the statement holds prior to time $0 < t < t_{p,K_r-K}$ where p is a process in $\overline{K_{r'}}$, and p receives only messages from itself at time t in e_r and f_1 . Then by the same arguments as in the base case together with the inductive hypothesis, these messages are received in both executions and leave the executions indistinguishable to p . So suppose the statement holds prior to time $0 < t < t_{p,K_r-K}$ for some process $p \in \overline{K_{r'}}$, and p receives a message m at time t in e_r or f_1 from another process.

Note that in case m is received in e_r , it is known by r at t_r : If $p \in K_r - K$, then p receives m in e_r before $t_{p,K_r-K} = t_{p,r}$, hence, by definition of $t_{p,r}$, m is known by r at t_r . Else, there exists a process $q \in K_r - K$ that knows about m in e_r before time $t_{q,r}$. This means that m 's receipt happened-before q 's receipt of some message m_q from another process, which occurred before time $t_{q,r}$. By definition of $t_{q,r}$, m_q is known by r at t_r , which means that m_q 's receipt happened-before some message receipt at r occurring up to time t_r . Overall, m 's receipt happened-before some message receipt at r occurring up to time t_r . This means that m is known by r at t_r .

For messages received by p in f_1 or e_r before time t , we may apply the inductive hypothesis, which yields that f_1 is indistinguishable to p from e_r until the receipt time of any such message. Hence, f_1 and e_r are indistinguishable to p prior to t . To show that f_1 and e_r are indistinguishable to p until t (including), we will show that m is received in both executions at time t . This is enough as by assumption, no message other than m is received by p at t from other processes; and messages p possibly receives from itself after m at time t are received in both executions and leave the executions indistinguishable to p , by the same arguments as in the base case.

Let s be the sender of m . To apply the inductive hypothesis to messages received by s in either execution until the time in which s sends m , we need to show they satisfy the conditions of the induction's statement: $s \in \overline{K_{r'}}$, and these messages are received before time t_{s,K_r-K} . For the latter, we show the stronger statement that s sends m before time t_{s,K_r-K} .

We start with the case in which m is received in e_r . As m is known by r at t_r (as shown above), $s \in K_r$ by Lemma B.10. We show that $s \notin K_r \cap K_{r'}$, and then $s \in K_r - (K_r \cap K_{r'}) = \overline{K_{r'}}$ will follow. As m is received by $p \neq s$ before t_{p,K_r-K} , there exists a process $q \in K_r - K$ that knows in e_r about m before time $t_{q,r}$. Intuitively, $s \notin K_r \cap K_{r'}$ is true because $q \notin K$, so when r decides in e_r it does not know that q knows about any message sent by a process in the intersection. Formally, as q knows about m before $t_{q,r}$, there exists a message m^* , received by q before time $t_{q,r}$, such that either $q = p$ and $m^* = m$, or $q \neq p$ and m^* 's receipt (and hence sending) happened-before the sending of m^* . In both cases, m^* is known by r at t_r as it is received by q before time $t_{q,r}$ from another process. Then by Lemma B.14, $s \in \text{known_acceptors}[q][q]$ right after handling m^* in Algorithm 8. Additionally, by condition (3) in Lemma B.9, after handling m^* , $\text{known_acceptors}[q][q] \subseteq \text{AlgReqs}[r][q]$. Therefore, $s \in \text{AlgReqs}[r][q]$. But since $q \notin K$, and $q \in K_r$, then by definition of K : $\text{AlgReqs}[r][q] \cap (K_r \cap K_{r'}) = \emptyset$. Therefore, $s \notin K_r \cap K_{r'}$. So indeed $s \in \overline{K_{r'}}$. We still need to show that s sends m before time t_{s,K_r-K} . If $t_{s,K_r-K} = t_{s,r}$, then as m is known by r at t_r (as shown above), s sends m before time t_{s,K_r-K} by Corollary B.8. Else, by definition of t_{s,K_r-K} , s receives from another process a message m_s at time t_{s,K_r-K} that is not known by any process $q' \in K_r - K$ before time $t_{q',r}$. Assume for sake of contradiction that s sends m at

time $\geq t_{s,K_r-K}$. By our assumption, s does not send messages right before receiving a message from another process at the exact same time. Namely, s sends m after receiving m_s . As m is known by q before time $t_{q,r}$, then m_s is as well, which is a contradiction. So in any case, s sends m before time t_{s,K_r-K} .

We proceed to the case that m is received in f_1 . m is received before time $t_{p,K_r-K} \leq t_{p,r} \leq t_r + \epsilon \leq t_d$ in f_1 . Then m is sent until time t_d . Messages sent until t_d to process $p \in \overline{K_{r'}}$ in f_1 are received by it only if they are from processes in $\overline{K_{r'}}$. Therefore, $s \in \overline{K_{r'}}$. In addition, m is sent before t_{s,K_r-K} , as any messages $s \in \overline{K_{r'}}$ sends in f_1 until time t_d at t_{s,K_r-K} or later are lost.

We conclude that we may apply the inductive hypothesis to messages received by s until the time in which it sends m . By the inductive hypothesis, f_1 and e_r are indistinguishable to s until receiving the last message (if any) in either execution up to the time in which it sends m . They hence remain indistinguishable to s from that moment (or from time 0 in case no message receipts by s precede the sending of m) until it sends m (in e_r or f_1). Therefore, m is sent in the other execution as well at the same time. Given that m originally appears in f_1 , it is not lost and reaches its target in e_r as well, since no messages to correct processes are lost in e_r . Given that m originally appears in e_r , it is not lost and reaches its target in f_1 as well, since m is a message from a process in $\overline{K_{r'}}$ to a process in $\overline{K_{r'}}$, reaching its target p at $t < t_{p,K_r-K}$. Furthermore, m reaches its target at the same time in both executions—according to the corresponding latency in $\mathcal{L}$. That is, m is received in both executions at time t and we are done. $\square$

Lemma B.22. For every $p \in K_r - K$: $f_1 \stackrel{p}{\approx}_{t_d} f_2$, for r chosen in the proof of Claim B.4, and t_d, K, f_1 and f_2 defined in that proof.

Proof. Let p be a process in $K_r - K$. By Lemma B.21, $f_1 \stackrel{p}{\approx}_{t_{p,r}} e_r$. In addition, it could be proven for f_2 , similarly to the proof of Lemma B.11 for e_2 , that $f_2 \stackrel{p}{\approx}_{t_{p,r}} e_r$.

Overall, we get $f_1 \stackrel{p}{\approx}_{t_{p,r}} f_2$. As p receives no messages at $t_{p,K_r-K} = t_{p,r}$ or later until t_d (including) in either f_1 or f_2 , $f_1 \stackrel{p}{\approx}_{t_d} f_2$. $\square$

Lemma B.23. $f_2 \stackrel{p}{\approx} f_1$, for f_1 and f_2 defined in the proof of Claim B.4, and any process $p \notin K$, where K is also defined in that proof.

Proof. For every $p \in K_r - K$: $f_1 \stackrel{p}{\approx}_{t_d} f_2$, by Lemma B.22. A similar proof would show that this holds also for every $p \in K_{r'} - K$. Any process $p \notin K_r \cup K_{r'}$ does not receive any messages in f_1 and f_2 until t_d , so $f_1 \stackrel{p}{\approx}_{t_d} f_2$ for it too. Overall, we have $f_1 \stackrel{p}{\approx}_{t_d} f_2$ for any $p \notin K$. The processes not in K are the ones which remain up after t_d , and message transmission after t_d is according to the latencies of $\mathcal{L}^*$ in both f_1 and f_2 , so the executions remain indistinguishable to these processes at all times. $\square$

The following lemmas are used in the proof of Claim B.5.

Lemma B.24. No process in $\mathcal{F}$ appears in $\text{AlgReqs}[r].\text{keys}()$ for any process r .

Proof. $\text{AlgReqs}[r]$ is computed by running Algorithm 8 over messages in e_r , computing `known_acceptors` and then setting $\text{AlgReqs}[r]$ to the value of `array_to_dict(known_acceptors[r])` after processing all messages known by r when r decides. A simple induction on the messages known by r when r decides shows that the keys in any `array_to_dict(known_acceptors[p])` for any process p are processes receiving these messages. As no process in $\mathcal{F}$ receives any message in e_r , we are done. $\square$

Lemma B.25. Consider a process $r \notin \mathcal{F}$. No DoAdopt message is sent in e_r^* , for e_r^* defined in the proof of Claim B.5.

Proof. DoAdopt messages are sent in KCensus on lines 26, 37 and 50.

First, we show that no DoAdopt message is sent in e_r^* on line 26. By Observation A.1, only value v_r is sent in `Accept&Spread` messages. Therefore, `accepted` on any process may only be set to v_r on line 24. Hence, when reaching line 25, `accepted` = v_r . Consequently, the condition on line 25 is never met, and line 26 is never reached in e_r^* .

Second, we show that no DoAdopt message is sent in e_r^* on line 50: By Lemma B.24, for any process p , all processes in $\text{AlgReqs}[p].\text{keys}()$ are correct. As the network is stable in e_r^* , failure detectors do not suspect any correct process as having crashed. Therefore, the failure detector on any process p may invoke `PotentialFailure` only with a process not in $\text{AlgReqs}[p].\text{keys}()$ as an argument, and then the condition on line 49 will not be met. Hence, line 50 is never reached in e_r^* .

Lastly, we show that no DoAdopt message is sent in e_r^* , including on line 37. Consider the first sending step of a DoAdopt or Freeze message in e_r^* . It cannot be a sending of a DoAdopt message, as we showed that no DoAdopt message is sent on lines 26 and 50, and a DoAdopt message is sent on line 37 only after a Freeze message is sent and received. It also cannot be a sending of a Freeze message, as a Freeze message is sent only on line 15 and only after a DoAdopt message is sent and received. Therefore, there is no sending step of a DoAdopt (or Freeze) message in e_r^* . $\square$

Lemma B.26. Consider a process $r \notin \mathcal{F}$. Any process $p \notin \mathcal{F}$ receives its first Accept&Spread message in e_r^* before or at the same time it receives its first message in e_r if any, for e_r^* defined in the proof of Claim B.5.

Proof. Denote the shortest weighted distance in the graph $\mathcal{L}$ from r to a correct process p by $\text{dist}_{\mathcal{L}}(r, p)$. We prove by induction on $\text{dist}_{\mathcal{L}}(r, p)$ that every process $p \notin \mathcal{F}$ receives its first Accept&Spread message in e_r^* at time $\leq \text{dist}_{\mathcal{L}}(r, p)$. Due to the assumption that a Propose invocation happened-before any message sending in Alg, p cannot receive a message in e_r at a time earlier than $\text{dist}_{\mathcal{L}}(r, p)$.

For the base case, process r has $\text{dist}_{\mathcal{L}}(r, r) = \mathcal{L}[r, r] = 0$. Indeed, r receives an Accept&Spread message from itself at time 0 in e_r^* .

Consider a correct process $p \neq r$. Suppose the statement holds for any correct process q such that $\text{dist}_{\mathcal{L}}(r, q) < \text{dist}_{\mathcal{L}}(r, p)$. Let P be a shortest path from r to p in $\mathcal{L}$, and let s be the process preceding p in this path. By the inductive hypothesis, s receives its first Accept&Spread message in e_r^* at time $\leq \text{dist}_{\mathcal{L}}(r, s)$. When handling its first Accept&Spread message, process s adds its id to its local known_acceptors[s] value on line 29, which triggers a sending of an Accept&Spread message m to p on line 33. This message reaches p at time $\leq \text{dist}_{\mathcal{L}}(r, s) + \mathcal{L}[s, p]$. As P is a shortest path in $\mathcal{L}$ from r to p , the weight of P is $\text{dist}_{\mathcal{L}}(r, p)$. P 's sub-path from r to s is a shortest path from r to s , hence its weight is $\text{dist}_{\mathcal{L}}(r, s)$. Overall, $\text{dist}_{\mathcal{L}}(r, p) = \text{dist}_{\mathcal{L}}(r, s) + \mathcal{L}[s, p]$. Therefore, message m reaches p at time $\leq \text{dist}_{\mathcal{L}}(r, p)$. $\square$

Lemma B.27. Consider a process $r \notin \mathcal{F}$. Let κ_{Alg}^t be the value of known_acceptors in Algorithm 8 after processing all messages, known by r at t_r , received in e_r up to some time t , and let $\kappa_{\text{KCensusAlgReqs}}^t[p]$ be the value of known_acceptors at some process p in Algorithm 3 right after time t during e_r^* , for e_r^* defined in the proof of Claim B.5.

Then for any time t , $\kappa_{\text{Alg}}^t[p][q] \subseteq \kappa_{\text{KCensusAlgReqs}}^t[p][q]$ for all processes p and q .

Proof. We prove by induction on message receipt times in e_r of messages, which are known by r at t_r , that for any such message receipt time t , $\kappa_{\text{Alg}}^t[p][q] \subseteq \kappa_{\text{KCensusAlgReqs}}^t[p][q]$ for all processes p, q . It is enough to consider times in which messages known by r at t_r are received in e_r , because these are the only times for which sets in κ_{Alg}^t might be expanded, and sets in $\kappa_{\text{KCensusAlgReqs}}^t$ may never shrink.

For the base case, consider time 0. Before any message is handled, all sets in known_acceptors in e_r are empty except for known_acceptors[r][r] which equals $\{r\}$. The only messages that may be received at time 0 in e_r are messages sent by r to itself, due to the assumption that a Propose invocation happened-before any message sending in Alg together with the fact that message transmissions between distinct processes take positive time. Processing these messages from r to itself does not change any value in known_acceptors. As for e_r^* , before any message is handled, all sets in known_acceptors are empty. r sends itself a message at time 0 in e_r^* upon proposing. Processing it adds r to known_acceptors[r][r] on line 29.

Suppose the statement holds until time $t^p > 0$ in which some process p receives from process s a message known by r at t_r in e_r . We need to show that $\kappa_{\text{Alg}}^{t^p}[u][v] \subseteq \kappa_{\text{KCensusAlgReqs}}^{t^p}[u][v]$ for all processes u and v . By the inductive hypothesis, $\kappa_{\text{Alg}}^t[u][v] \subseteq \kappa_{\text{KCensusAlgReqs}}^t[u][v]$ for any time $t < t^p$ and all processes u and v . The values of $\kappa_{\text{Alg}}^t[u]$ for any process $u \neq p$ do not change at time $t = t^p$ (assuming no two processes receive messages at the same time, otherwise we could apply the following proof to each of them). So it remains to show that the containment still holds after values are possibly added to cells in $\kappa_{\text{Alg}}^t[p]$ upon processing messages received by p at time t^p .

If p receives any messages from itself at time t^p in e_r , then the only value possibly added to cells in $\kappa_{\text{Alg}}^t[p]$ upon processing these messages is p added to $\kappa_{\text{Alg}}^t[p][p]$ on line 7. We next show that p is already in $\kappa_{\text{Alg}}^t[p][p]$ before processing these messages, hence no values are added to $\kappa_{\text{Alg}}^t[p][p]$ upon processing these messages. If $p = r$, then $r \in \kappa_{\text{Alg}}^t[r][r]$ since initialization. Else ($p \neq r$), p must receive a message from another process prior to sending itself a message and receiving it, due to the assumption that a Propose invocation happened-before any message sending in Alg. When processing the message from the other process, p adds its id to $\kappa_{\text{Alg}}^t[p][p]$ on line 7.

So it remains to consider a message m , known by r at t_r in e_r , that p receives from another process s at time t^p . Let t^s be the time s sends m . Then $t^s = t^p - \mathcal{L}[s, p]$. The values possibly added to cells in $\kappa_{\text{Alg}}^t[p]$ upon processing m are: (a) p to $\kappa_{\text{Alg}}^{t^p}[p][p]$ on line 7, (b) $\kappa_{\text{Alg}}^{t^s}[s][s]$ to $\kappa_{\text{Alg}}^{t^p}[p][p]$ on line 7, and (c) $\kappa_{\text{Alg}}^{t^s}[s][q]$ to $\kappa_{\text{Alg}}^{t^p}[p][q]$ for any process q on line 9. We next show that (1) $p \in \kappa_{\text{KCensusAlgReqs}}^{t^p}[p][p]$, (2) $\kappa_{\text{Alg}}^{t^s}[s][s] \subseteq \kappa_{\text{KCensusAlgReqs}}^{t^p}[p][p]$, and (3) $\kappa_{\text{Alg}}^{t^s}[s][q] \subseteq \kappa_{\text{KCensusAlgReqs}}^{t^p}[p][q]$ for any process q .

Let t_2 be the first time at which $\kappa_{\text{KCensusAlgReqs}}^t[s]$ gained the set of values it had right after time $t = t^s$. So t_2 could either be at initialization before receiving any Accept&Spread message, or when s receives some Accept&Spread message in e_r^* , as $\kappa_{\text{KCensusAlgReqs}}^t[s]$ is expanded only during handling Accept&Spread messages on lines 23–33. We show below that the first alternative is impossible, as s receives its first Accept&Spread message at some time $t_1 \leq t^s$ and expands $\kappa_{\text{KCensusAlgReqs}}^t[s]$ upon handling this message. Therefore, the second alternative is the correct one. Namely, s receives an Accept&Spread message at time

t_2 in e_r^* . Upon processing it, s sends to p an $\text{Accept\&Spread}(v_r, \kappa_{KCensusAlgReqs}^{t^s}[s])$ message on line 33. p receives this message at time $t_2 + \mathcal{L}[s, p] \leq t^s + \mathcal{L}[s, p] = t^p$. When handling it, on line 29 p extends $\kappa_{KCensusAlgReqs}^t[p][p]$ to contain the processes in $\{p\} \cup \kappa_{KCensusAlgReqs}^{t^s}[s][s]$ at time $t = t^p$. Hence, condition (1) above is satisfied, and condition (2) is too: $\kappa_{KCensusAlgReqs}^{t^s}[s][s]$ is a superset of $\kappa_{Alg}^{t^s}[s][s]$ by the inductive hypothesis, which is applicable to t^s as $t^s < t^p$ (since $\mathcal{L}[s, p] > 0$). Additionally, on line 31 p extends $\kappa_{KCensusAlgReqs}^t[p][q]$ to contain $\kappa_{KCensusAlgReqs}^{t^s}[s][q]$ at time $t = t^p$ for each process q . Hence, condition (3) above is satisfied: $\kappa_{KCensusAlgReqs}^{t^s}[s][q]$ is a superset of $\kappa_{Alg}^{t^s}[s][q]$ by the inductive hypothesis.

We show the missing part: that s receives its first $\text{Accept\&Spread}$ message in e_r^* at some time $t_1 \leq t^s$, and expands $\kappa_{KCensusAlgReqs}^t[s]$ upon handling this message. If $s \neq r$, then s received a message in e_r at time $\leq t^s$, due to the assumption that a Propose invocation happened before any message sending in Alg. Thus, by Lemma B.26, s receives its first $\text{Accept\&Spread}$ message in e_r^* at some time $t_1 \leq t^s$. The latter is correct also if $s = r$, as r receives its first $\text{Accept\&Spread}$ message in e_r^* at time 0, upon sending such a message to itself on line 10. The value of $\kappa_{KCensusAlgReqs}^t[s][s]$ is $\emptyset$ before s handles any $\text{Accept\&Spread}$ message in e_r^* . When s receives its first $\text{Accept\&Spread}$ message in e_r^* at time t_1 , it adds s to $\kappa_{KCensusAlgReqs}^{t_1}[s][s]$ on line 29. $\square$

Lemma B.28. *Consider a process $r \notin \mathcal{F}$. Calling $\text{can_commit}(v_r)$ in e_r^* by r at time t_r would return True, for e_r^* defined in the proof of Claim B.5.*

Proof. accepted is set to v_r at r , as upon proposing, r handles an $\text{Accept\&Spread}(v_r, \emptyset)$ message from itself. In addition, $\text{AlgReqs}[r] = \text{array_to_dict}(\text{known_acceptors}[r])$ for the value of known_acceptors in Algorithm 8 after processing all messages received in e_r which are known by r at t_r . Applying Lemma B.27 to time t_r and process r and every process $q \in \text{AlgReqs}[r].\text{keys}()$ yields $\text{AlgReqs}[r][q] \subseteq \kappa_{KCensusAlgReqs}^{t_r}[r][q]$, for $\kappa_{KCensusAlgReqs}^{t_r}[r]$ defined in the proof of Lemma B.27. QED follows. $\square$

C Fast-Path Correctness

In this section we prove that the validity and compatibility conditions, appearing in Section 3.1, are an exact characterization of correct fast paths.

We call the *fast path* of a protocol Alg the set of its executions in a stable network without contention, and we say that a recovery rule *preserves fast decisions* if, in every execution in which some process decides a value v on the fast path, the rule adopts v . Alg's fast path is *correct* if such a rule exists.

Both proofs reuse the construction from the proof of Theorem B.1: for each correct process r , it takes the single-proposer execution e_r in a stable network, collects the messages r knows when it decides, and processes them with Listing 8 to obtain $\text{AlgReqs}[r]$, whose keys are the processes whose acceptance r knows of when it decides. Claim B.2 shows that the resulting combination has the required normal form, whatever Alg does; when Alg is a correct consensus protocol, Claim B.3 shows that each requirement is valid and Claim B.4 that every pair is compatible.

Theorem C.1 (Sufficiency). *Any fast path satisfying the validity and compatibility conditions could leverage the knowledge derivable from its communication to admit a recovery rule that preserves fast decisions in every execution.*

Proof. Let Alg be any deterministic consensus protocol with a fast path which is taken in executions with a stable network and no contention, such that this fast path satisfies the validity and compatibility conditions. Derive AlgReqs for Alg as above; being derived in a stable network without contention, it reflects the requirements of Alg's fast path. Here Alg is not assumed correct, so validity and compatibility come from the conditions rather than from Claims B.3 and B.4: the requirement $\text{AlgReqs}[r]$ of each process r is valid because Alg's fast-path quorums satisfy the validity condition; and the combination of requirements AlgReqs is compatible since Alg's fast path satisfies the compatibility condition. In addition, by Claim B.2, AlgReqs has the normal form required by KCensus. Hence, KCensus could use AlgReqs . For each single-proposer execution, we alter KCensus to disseminate messages using the same communication pattern as Alg's fast path in its single-proposer execution with the same proposer. The resulting consensus protocol obtains the knowledge derivable from the communication patterns of Alg, and its adoption rule preserves fast decisions in every execution by KCensus's agreement (Theorem A.22). $\square$

Theorem C.2 (Necessity). *The requirements derived from the communication of any correct fast path satisfy the validity and compatibility conditions, even if this fast path is not defined in terms of acceptance.*

Proof. Let Alg be any deterministic consensus protocol with a correct fast path which is taken in executions with a stable network and no contention. As its fast path is correct, Alg has a safe recovery rule. We next show that the requirements derived from Alg's fast-path communication satisfy the validity and compatibility conditions.

Derive $AlgReqs$ for Alg as above. By Claim B.3, the requirement $AlgReqs[r]$ of each process r is valid. This means that Alg's fast-path quorum contains more than f processes. Namely, the validity condition is satisfied.

By Claim B.4, the combination of requirements $AlgReqs$ is compatible. This means that for every pair of proposers p and q with quorums $Q_p = AlgReqs[p].keys()$ and $Q_q = AlgReqs[q].keys()$, at least $f + 1$ processes received a chain of messages from some process in $Q_p \cap Q_q$, sent after it learned of the proposal, as known by p or by q when deciding in its single-proposer execution. Namely, the requirements derived from Alg's fast-path communication satisfy the compatibility condition. $\square$